

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng DEX aggregator sử dụng kiến trúc intent solver

VŨ TIẾN AN NGUYỄN

Nguyen.VTA225148@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: TS. Trần Vĩnh Đức

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước hết, tôi xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn, người đã định hướng đề tài, góp ý về cách tiếp cận bài toán và chỉ ra những thiếu sót trong quá trình tôi xây dựng hệ thống. Những trao đổi trong các buổi gặp đã giúp tôi nhìn nhận lại nhiều quyết định thiết kế mà nếu tự làm một mình, tôi khó có thể nhận ra.

Tôi cũng xin cảm ơn các thầy cô trong Khoa Kỹ thuật Máy tính đã truyền đạt những kiến thức nền tảng trong suốt quá trình học tập tại trường, đó là cơ sở để tôi có thể tiếp cận và hoàn thành đồ án này.

Cuối cùng, tôi xin cảm ơn gia đình và bạn bè đã luôn động viên, tạo điều kiện để tôi tập trung hoàn thành đồ án tốt nghiệp trong khoảng thời gian vừa qua. Do thời gian và kiến thức còn hạn chế, đồ án chắc chắn không tránh khỏi thiếu sót, tôi rất mong nhận được sự góp ý từ thầy cô để hoàn thiện hơn.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Các sàn giao dịch phi tập trung (DEX) hiện nay phần lớn vận hành dựa trên cơ chế tạo lập thị trường tự động (AMM), trong đó giá tài sản được xác định theo công thức toán học dựa trên lượng thanh khoản trong hồ chứa. Cách làm này khiến các giao dịch khối lượng lớn dễ gặp hiện tượng trượt giá và tạo cơ hội cho các tác nhân khai thác giá trị có thể trích xuất tối đa (MEV) thu lợi từ chính giao dịch đó. Một số giao thức đã chuyển sang mô hình giao dịch dựa trên ý định, trong đó người dùng ký lệnh và giao việc tìm đối tác, thực thi cho các bên thứ ba gọi là filler, tiêu biểu là UniswapX và CoW Protocol. Tuy nhiên, qua khảo sát có thể thấy phần lớn các giao thức này vẫn xử lý lệnh theo nguyên tắc tất cả hoặc không có gì, chưa ràng buộc trách nhiệm của filler một cách chặt chẽ, và phần lớn hoạt động trong phạm vi một chuỗi.

Trước thực trạng đó, đồ án lựa chọn hướng tiếp cận giao dịch dựa trên ý định làm nền tảng, đồng thời bổ sung các cơ chế còn thiếu ở các hệ thống đã khảo sát. Hướng đi này cho phép tách riêng việc thể hiện mong muốn giao dịch khỏi việc thực thi, qua đó giảm chi phí cho người dùng và tạo điều kiện tối ưu việc thực thi thông qua cạnh tranh giữa nhiều filler.

Trên cơ sở đó, đồ án xây dựng hệ thống NeutronX gồm bốn cơ chế chính. Thứ nhất là khớp lệnh từng phần theo đấu giá kiểu Hà Lan, kết hợp chuẩn ký dữ liệu EIP-712 và Permit2. Thứ hai là cơ chế đặt cọc động ràng buộc trách nhiệm filler thông qua phạt. Thứ ba là một thành phần thực thi dự phòng đảm bảo lệnh luôn được hoàn tất trước hạn. Thứ tư là phần mở rộng cho phép giao dịch xuyên chuỗi giữa hai mạng tương thích EVM bằng hợp đồng khóa thời gian theo mã băm, có hỗ trợ khớp từng phần thông qua cây Merkle.

Đóng góp chính của đồ án là đã thiết kế, cài đặt và kiểm thử thành công một hệ thống tổng hợp DEX hoàn chỉnh trên môi trường mạng giả lập. Trong hệ thống này, lệnh được khớp từng phần một cách minh bạch, filler chịu trách nhiệm về cam kết của mình, lệnh luôn được đảm bảo thực thi trước thời hạn, và người dùng có thể thực hiện giao dịch hoán đổi xuyên chuỗi mà không cần thông qua cầu nối trung gian.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu, phạm vi và giới hạn.....	2
1.2.1 Mục tiêu.....	2
1.2.2 Phạm vi.....	2
1.2.3 Giới hạn	3
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	3
CHƯƠNG 2. BỐI CẢNH VÀ ĐỀ XUẤT GIẢI PHÁP.....	4
2.1 Quá trình phát triển của Uniswap và những hạn chế hiện tại	4
2.2 Tổng quan hệ thống NeutronX và mô hình lệnh	5
2.3 Khớp lệnh từng phần theo đấu giá kiểu Hà Lan	5
2.4 Đặt cọc và phạt.....	6
2.5 Cơ chế dự phòng.....	7
2.6 Giao dịch xuyên chuỗi không sử dụng cầu nối	8
2.7 Đánh đổi trong thiết kế.....	10
CHƯƠNG 3. XÂY DỰNG CÁC HỢP ĐỒNG THÔNG MINH.....	11
3.1 Kiến trúc tổng thể và luồng giao dịch	11
3.2 Mô hình dữ liệu lệnh.....	13
3.3 Hợp đồng PartialFillReactor	14
3.4 Hợp đồng FillAuction	16
3.5 Hợp đồng FallbackExecutor.....	18
3.6 Các hợp đồng cho giao dịch xuyên chuỗi	19
3.7 Backend và Frontend.....	23

CHƯƠNG 4. KIỂM THỬ VÀ RÀ SOÁT BẢO MẬT	26
4.1 Phương pháp và môi trường kiểm thử	26
4.2 Các tính chất đã chứng minh.....	27
4.2.1 Khớp lệnh từng phần luôn đúng và không bị kẹt.....	27
4.2.2 Người dùng không bao giờ bị trả thiếu	28
4.2.3 Cơ chế cọc và phạt không bị khai thác kinh tế.....	29
4.2.4 Chống trích xuất giá trị bằng MEV	30
4.2.5 Ngăn khớp trùng và bảo đảm hoàn tất lệnh	31
4.2.6 Xuyên chuỗi: chống griefing và an toàn về thời gian.....	31
4.3 Độ phủ và phạm vi kiểm thử.....	32
4.4 Rà soát bảo mật bởi Trufy	32
4.4.1 Tổng quan phát hiện	32
4.4.2 Phát hiện đã khắc phục	33
4.4.3 Các phát hiện cần môi trường production để kiểm chứng	34
4.5 Kết luận chương	35
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	36
5.1 Kết luận	36
5.2 Hạn chế	36
5.3 Hướng phát triển.....	37
TÀI LIỆU THAM KHẢO.....	39
PHỤ LỤC.....	41
A. MỘT SỐ USE CASE KHÁC	41
A.1 Cấp quyền chi tiêu một lần qua Permit2	41
A.2 Kiểm tra đăng ký hợp lệ của filler	41
A.3 Truy vấn giá thị trường	42
A.4 Gợi ý tham số lệnh tự động	43

B. CƠ SỞ DỮ LIỆU	45
B.1 Bảng orders (lệnh trong cùng chuỗi)	45
B.2 Bảng fills (các lần khớp).....	45
B.3 Các bảng cho giao dịch xuyên chuỗi	45
C. DANH SÁCH TRƯỜNG HỢP KIỂM THỬ	47
D. KIẾN THỨC NỀN TẢNG	50
D.1 Blockchain và hợp đồng thông minh	50
D.2 Nonce	50
D.3 Cây Merkle	50
D.4 Hợp đồng khóa thời gian theo mã băm (HTLC)	51
D.5 Chuẩn ký EIP-712 và Permit2	51
D.6 Đấu giá kiểu Hà Lan	51
D.7 MEV và tấn công chèn lệnh	51
D.8 CREATE2.....	52
D.9 Các thư viện OpenZeppelin sử dụng	52
E. BÁO CÁO RÀ SOÁT BẢO MẬT (TRUFY)	53

DANH MỤC HÌNH VẼ

Hình 2.1	Đường cong giá giảm dần theo block trong đấu giá kiểu Hà Lan và các điểm khớp lệnh từng phần.	6
Hình 2.2	Cách phân biệt hoàn cọc (filler không có lỗi) hay phạt cọc (filler bỏ dở) sau thời hạn.	7
Hình 2.3	Kiến trúc giao dịch xuyên chuỗi với một hợp đồng ký quỹ riêng cho mỗi slot trên cả hai chuỗi.	8
Hình 2.4	Cách tạo bí mật cho từng slot từ một bí mật gốc và cây Merkle tương ứng.	9
Hình 3.1	Kiến trúc tổng thể của hệ thống NeutronX gồm bốn tầng.	11
Hình 3.2	Sơ đồ gói các hợp đồng thông minh của NeutronX.	12
Hình 3.3	Sơ đồ phụ thuộc giữa các hợp đồng.	12
Hình 3.4	Sơ đồ trình tự vòng đời một lệnh được khớp từng phần trong cùng một chuỗi.	13
Hình 3.5	Các trạng thái của một lệnh trong hợp đồng PartialFillReactor.	16
Hình 3.6	Sơ đồ trình tự luồng giao dịch xuyên chuỗi (6 bước).	22
Hình 3.7	Ràng buộc thời gian $T2 < T1$ bảo vệ filler khỏi mất cả hai đầu.	23
Hình 3.8	Một số màn hình của giao diện người dùng.	24
Hình 3.9	Giao diện người dùng cho giao dịch xuyên chuỗi.	24

DANH MỤC BẢNG BIỂU

Bảng 3.1	Các trường của một lệnh <code>PartialFillOrder</code>	14
Bảng 3.2	Các hàm chính của <code>PartialFillReactor</code>	14
Bảng 3.3	Các hàm chính của <code>FillAuction</code>	17
Bảng 3.4	Bảng hoàn cọc mặc định (đơn vị: phần vạn của số cọc), theo cỡ lệnh và tỉ lệ giao so với cam kết.	18
Bảng 3.5	Hàm chính của <code>FallbackExecutor</code>	19
Bảng 3.6	Các hàm chính của nhóm hợp đồng xuyên chuỗi.	20
Bảng 3.7	So sánh hai cách làm xuyên chuỗi.	21
Bảng 4.1	Tổng hợp kết quả <code>forge test</code> : 149/149 trường hợp đạt, nhóm theo thành phần.	26
Bảng 4.2	Tám phát hiện từ báo cáo Trufy và trạng thái xử lý.	33
Bảng 5.1	Đối chiếu kết quả với các mục tiêu ban đầu.	36
Bảng D.1	Các thư viện <code>OpenZeppelin</code> được sử dụng trong đồ án.	52

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AMM	Cơ chế tạo lập thị trường tự động (Automated Market Maker)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
DeFi	Tài chính phi tập trung (Decentralized Finance)
DEX	Sàn giao dịch phi tập trung (Decentralized Exchange)
EIP-712	Chuẩn ký dữ liệu có cấu trúc trên Ethereum (Ethereum Improvement Proposal 712)
ERC-20	Tiêu chuẩn token thay thế được trên Ethereum (Ethereum Request for Comment 20)
EVM	Máy ảo Ethereum (Ethereum Virtual Machine)
HTLC	Hợp đồng khóa thời gian theo mã băm (Hash Time Lock Contract)
MEV	Giá trị có thể trích xuất tối đa (Maximal Extractable Value)
RPC	Lời gọi thủ tục từ xa (Remote Procedure Call)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Vài năm gần đây, tài chính phi tập trung (Decentralized Finance - DeFi) [1] phát triển nhanh chóng, và một trong những thành phần quan trọng nhất của nó là các sàn giao dịch phi tập trung (Decentralized Exchange - DEX). Khác với sàn giao dịch tập trung, DEX cho phép người dùng đổi tài sản số trực tiếp với nhau thông qua hợp đồng thông minh, mà không phải gửi tài sản cho một bên trung gian giữ hộ. Hiện nay khối lượng giao dịch mỗi ngày trên các DEX lớn như Uniswap đã lên tới hàng tỉ đô la, cho thấy mô hình này ngày càng được sử dụng nhiều.

Nếu nhìn lại quá trình phát triển của DEX, có thể thấy nó được cải tiến dần qua từng giai đoạn. Những DEX đầu tiên, tiêu biểu là Uniswap, hoạt động theo cơ chế tạo lập thị trường tự động (Automated Market Maker - AMM). Với cơ chế này, người dùng giao dịch trực tiếp với một hồ thanh khoản (liquidity pool), giá được tính theo công thức dựa trên lượng tài sản trong hồ. Mô hình này đơn giản và luôn sẵn sàng giao dịch, nhưng có một điểm yếu: giá người dùng nhận được phụ thuộc hoàn toàn vào lượng thanh khoản hiện có trong hồ trên chuỗi. Khi giao dịch một khối lượng lớn, giá sẽ bị lệch đáng kể so với mong đợi (hiện tượng trượt giá), và điều này còn tạo cơ hội cho một số bên lợi dụng để kiếm lời từ chính giao dịch của người dùng (hiện tượng MEV [2], trình bày thêm ở Phụ lục D.7).

Để khắc phục, gần đây xuất hiện một hướng tiếp cận mới gọi là giao dịch dựa trên ý định (intent-based). Ở hướng này, người dùng không tự thực hiện giao dịch mà ký một “ý định” mô tả tài sản muốn đổi. Việc tìm nguồn thanh khoản và thực hiện giao dịch do các bên thứ ba (gọi là filler) cạnh tranh nhau đảm nhận. Ưu điểm của hướng này là filler không bị giới hạn trong một hồ thanh khoản nào; họ có thể tập hợp thanh khoản từ nhiều nguồn khác nhau, kể cả các nguồn ngoài chuỗi (off-chain), nên người dùng có cơ hội nhận được giá tốt hơn so với khi giao dịch trong phạm vi một hồ AMM.

Tuy nhiên, khi tìm hiểu các hệ thống đang có (trình bày kỹ hơn ở Chương 2), có thể thấy ba vấn đề chưa được giải quyết. Thứ nhất, hầu hết các hệ thống khớp lệnh theo nguyên tắc toàn bộ hoặc không có gì (all-or-nothing), tức là khớp trọn vẹn cả lệnh hoặc không khớp gì, mà chưa hỗ trợ khớp từng phần. Thứ hai, filler đăng ký nhận lệnh nhưng nếu sau đó không thực hiện thì cũng không chịu chế tài nào, nên người dùng có thể phải chờ lâu hoặc lệnh bị bỏ dở cho tới khi hết hạn. Thứ ba, việc giao dịch giữa hai chuỗi khác nhau (xuyên chuỗi) thường phải đi qua cầu nối (bridge), mà cầu nối là nơi giữ tài sản của người dùng nên dễ trở thành mục tiêu

tần công.

Từ đó, đề án đặt mục tiêu xây dựng một hệ thống khớp lệnh từng phần. Hệ thống buộc filler đặt cọc và chịu phạt nếu bỏ dở, đảm bảo lệnh luôn được hoàn tất trước hạn, đồng thời hỗ trợ giao dịch xuyên chuỗi không cần cầu nối.

1.2 Mục tiêu, phạm vi và giới hạn

1.2.1 Mục tiêu

Đề án đặt ra năm mục tiêu, tất cả đều nằm ở tầng hợp đồng thông minh - phần lõi của hệ thống:

- Xây dựng cơ chế khớp lệnh **từng phần** dựa trên **đấu giá kiểu Hà Lan**, trong đó giá giảm dần theo từng khối để hạn chế bị khai thác MEV.
- Cho phép người dùng ký lệnh ngoài chuỗi bằng chuẩn **EIP-712** kết hợp **Permit2**, nhờ đó không tốn phí gas khi tạo lệnh.
- Xây dựng cơ chế **đặt cọc và phạt** để ràng buộc trách nhiệm của filler khi nhận lệnh.
- Xây dựng một **cơ chế dự phòng** để chắc chắn phần lệnh còn lại luôn được khớp trước khi hết hạn.
- Hỗ trợ **giao dịch xuyên chuỗi** giữa hai mạng tương thích EVM mà không dùng cầu nối, đồng thời vẫn cho phép khớp từng phần.

1.2.2 Phạm vi

Đề án tập trung chủ yếu vào tầng **hợp đồng thông minh**, vì đây là nơi chứa toàn bộ logic quan trọng của hệ thống. Phần backend và frontend được xây dựng ở mức đủ để chạy thử và minh họa, không hướng tới một sản phẩm hoàn chỉnh. Tài sản giao dịch được giới hạn ở các token theo chuẩn ERC-20. Hệ thống được triển khai và kiểm thử trên mạng giả lập cục bộ tạo bằng Anvil: một mạng sao chép trạng thái từ Ethereum mainnet cho trường hợp giao dịch trong cùng một chuỗi, và hai mạng giả lập riêng cho trường hợp xuyên chuỗi.

Phần chiến lược tính lợi nhuận của filler không được xử lý trong đề án. Trong thực tế, filler có thể là các nhà giao dịch, nhóm nghiên cứu hoặc công ty hoạt động độc lập, và họ thường không công khai mã nguồn cũng như cách quyết định có tham gia khớp lệnh hay không. Vì vậy, đề án tập trung xây dựng các quy tắc trên hợp đồng (đặt cọc, phạt, hoàn cọc) để ràng buộc filler, còn việc filler tối ưu lợi nhuận nằm ngoài phạm vi nghiên cứu. Hai filler kèm theo trong đề án là các chương trình mô phỏng đơn giản dùng để chạy thử các luồng giao dịch, không phải filler thực tế.

1.2.3 Giới hạn

Đồ án hiện được chạy và kiểm thử trên mạng giả lập, chưa triển khai lên mạng chính thức (mainnet). Để vận hành trên thực tế, hệ thống cần thêm nhiều điều kiện mà đồ án chưa có: thanh khoản thực, filler thực, một đợt kiểm toán bảo mật do chuyên gia thực hiện, cùng chi phí gas tương ứng. Ngoài ra, một số phần trong thiết kế vẫn mang tính tập trung, ví dụ khóa đồng ký (cosigner) hiện do một bên duy nhất giữ. Các giới hạn này được trình bày chi tiết hơn ở Chương 5, kèm theo hướng khắc phục.

1.3 Định hướng giải pháp

Đồ án chọn hướng giao dịch dựa trên ý định: người dùng (swapper) ký một lệnh mô tả muốn đổi gì, còn việc tìm thanh khoản và thực hiện được giao cho các filler cạnh tranh nhau đảm nhận. Cách này tách “ý muốn” khỏi “thực hiện”, giúp giảm chi phí tạo lệnh và cải thiện kết quả nhờ cạnh tranh.

Dựa trên hướng đó, đồ án xây dựng một hệ thống đặt tên là **NeutronX**, gồm bốn thành phần làm việc cùng nhau: tầng hợp đồng thông minh đảm nhận việc kiểm tra và thực hiện lệnh trên chuỗi; tầng backend đảm nhận việc đồng ký lệnh và theo dõi trạng thái; tầng filler đóng vai các bên khớp lệnh; và giao diện người dùng để tạo và theo dõi lệnh. Kiến trúc chi tiết của hệ thống được trình bày ở mục 3.1.

Đóng góp chính gồm bốn cơ chế hợp đồng: khớp lệnh từng phần theo đầu giá Hà Lan để hạn chế MEV; đặt cọc và phạt để ràng buộc filler; cơ chế dự phòng đảm bảo lệnh luôn hoàn tất; và giao dịch xuyên chuỗi không cần nối với khớp từng phần. Đây là sản phẩm thử nghiệm, kiểm chứng trên môi trường giả lập, không nhằm cạnh tranh với các hệ thống thương mại đang vận hành.

1.4 Bố cục đồ án

Phần còn lại được tổ chức như sau. Chương 2 khảo sát DEX qua Uniswap, chỉ ra chỗ còn thiếu và trình bày ý tưởng NeutronX. Chương 3 trình bày thiết kế và cài đặt chi tiết tầng hợp đồng, kèm tổng quan backend và frontend. Chương 4 trình bày kiểm thử và rà soát bảo mật. Chương 5 tổng kết kết quả, nêu hạn chế và đề xuất hướng phát triển.

CHƯƠNG 2. BỐI CẢNH VÀ ĐỀ XUẤT GIẢI PHÁP

Chương này gồm hai phần. Phần đầu tìm hiểu quá trình phát triển của Uniswap — sàn DEX lớn nhất hiện nay — để làm rõ những hạn chế còn tồn tại, từ đó xác định hướng giải pháp. Phần sau trình bày tổng quan hệ thống NeutronX cùng bốn cơ chế đóng góp ở mức ý tưởng, kèm lý do vì sao thiết kế như vậy; cách cài đặt cụ thể được trình bày ở Chương 3.

2.1 Quá trình phát triển của Uniswap và những hạn chế hiện tại

Uniswap là sàn giao dịch phi tập trung có khối lượng giao dịch lớn nhất hiện nay, với lịch sử phát triển lâu dài qua nhiều giai đoạn gắn liền với sự trưởng thành của lĩnh vực DEX. Do đó, đồ án chọn Uniswap làm điểm xuất phát để phân tích. Có thể chia quá trình phát triển này thành ba giai đoạn.

Giai đoạn 1 - Uniswap v2/v3 (AMM). Đây là mô hình tạo lập thị trường tự động: người dùng giao dịch trực tiếp với một hồ thanh khoản trên chuỗi, giá được tính theo công thức dựa trên lượng tài sản trong hồ. Ưu điểm là luôn sẵn sàng và hoàn toàn phi tập trung. Nhược điểm đã được nêu ở mục 1.1: giá người dùng nhận được bị giới hạn bởi thanh khoản có sẵn trên chuỗi, nên dễ bị trượt giá khi giao dịch lớn.

Giai đoạn 2 - UniswapX (giao dịch theo ý định). Để giải quyết vấn đề trên, Uniswap đưa ra UniswapX theo mô hình ý định: người dùng ký lệnh ngoài chuỗi, rồi các filler cạnh tranh nhau thực hiện. Vì filler có thể lấy thanh khoản từ nhiều nguồn, kể cả ngoài chuỗi, nên người dùng có thể nhận được giá tốt hơn. UniswapX cũng dùng đấu giá kiểu Hà Lan (giá giảm dần để filler tham gia khi có lãi) và bước đầu hỗ trợ giao dịch xuyên chuỗi. Cần lưu ý rằng phần lớn ý tưởng nền tảng mà đồ án dựa vào đã có từ giai đoạn này.

Giai đoạn 3 - những điểm chưa được giải quyết. Dù vậy, các hệ thống theo ý định hiện nay (kể cả UniswapX) vẫn tồn tại một số hạn chế chưa được giải quyết:

- **Khớp lệnh từng phần với giá được ký:** cho phép một lệnh được nhiều filler khớp dần thành nhiều phần, mỗi phần tính giá theo đường cong đã được xác thực, thay vì khớp trọn vẹn hoặc bỏ.
- **Ràng buộc filler bằng kinh tế:** buộc filler đặt cọc khi nhận lệnh và chịu phạt nếu bỏ dở phần đã nhận, thay vì cho filler tham gia mà không phải chịu trách nhiệm nào.
- **Xuyên chuỗi khớp từng phần, không cầu nối:** chia một lệnh xuyên chuỗi thành nhiều phần độc lập bằng hợp đồng khóa thời gian theo mã băm kết hợp

cây Merkle, thay vì khớp nguyên lệnh qua một cầu nối trung gian.

Ba điểm này là hướng mà đồ án tập trung giải quyết và sẽ được trình bày cụ thể từ mục 2.3 trở đi.

2.2 Tổng quan hệ thống NeutronX và mô hình lệnh

Ý tưởng chính của NeutronX là tách biệt hai vai trò: người dùng (swapper) ký lệnh biểu đạt mong muốn giao dịch; các filler cạnh tranh đặt cọc và thực hiện từng phần trên chuỗi dưới sự ràng buộc của các hợp đồng thông minh.

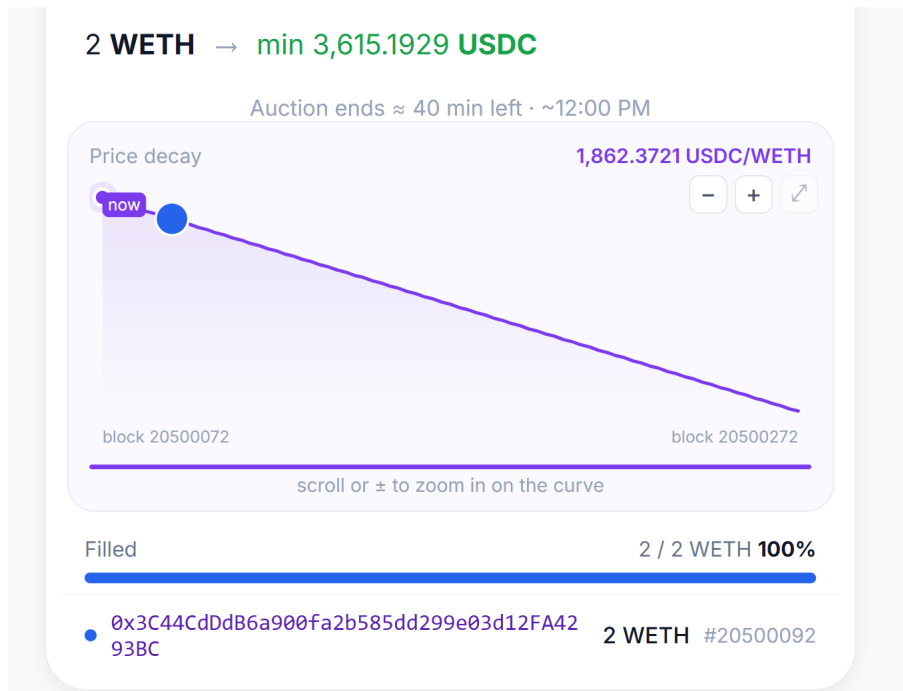
Toàn bộ thông tin của một giao dịch được gói trong một cấu trúc lệnh duy nhất (gọi là `PartialFillOrder`), gồm các trường mô tả tài sản vào và ra, khối lượng, mức đầu ra tối thiểu mà người dùng chấp nhận, thời hạn, cùng các tham số của đường cong giá đầu giá Hà Lan. Lệnh được ký theo chuẩn EIP-712 nên người dùng không tốn phí gas khi tạo lệnh (chi tiết EIP-712 và Permit2 xem Phụ lục D.5); danh sách đầy đủ các trường được trình bày ở mục 3.2.

Bốn mục tiếp theo trình bày bốn cơ chế đóng góp, tương ứng với ba điểm còn thiếu đã nêu ở mục 2.1 cùng với cơ chế dự phòng.

2.3 Khớp lệnh từng phần theo đầu giá kiểu Hà Lan

Cơ chế đầu tiên cho phép một lệnh được khớp thành nhiều phần. Mỗi lệnh gắn với một đường cong giá theo kiểu đầu giá Hà Lan: giá bắt đầu ở mức cao rồi giảm dần theo từng khối (block). Khi mức giá hiện tại đủ để có lãi, một filler sẽ khớp một phần khối lượng; phần còn lại tiếp tục có thể được khớp ở mức giá giảm dần cho đến khi khớp hết hoặc lệnh hết hạn. Cơ chế đầu giá kiểu Hà Lan giúp thị trường tự xác định mức giá mà tại đó có filler sẵn sàng tham gia (khái niệm đầy đủ xem Phụ lục D.6).

Thiết kế gồm hai đặc điểm quan trọng. Thứ nhất, mỗi phần được tính giá độc lập tại đúng mức giá ở thời điểm khớp, nhằm tránh việc tích lũy giá trị trên toàn bộ lệnh dẫn đến trần số ở phần cuối. Thứ hai, các tham số của đường cong giá nằm trong phần dữ liệu được ký, nên cũng được bên đồng ký (cosigner) xác thực. Bước xác thực này là bắt buộc: nếu không, một filler có thể tự đặt giá khởi điểm ở mức rất thấp rồi chiếm đoạt tài sản của người dùng.

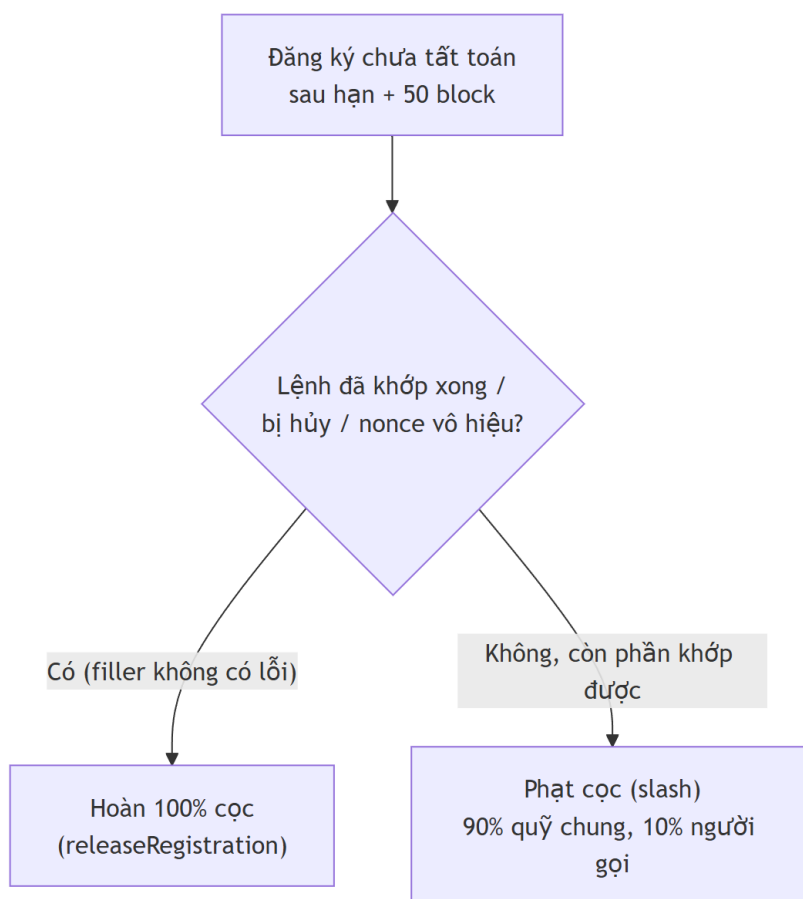


Hình 2.1: Đường cong giá giảm dần theo block trong đấu giá kiểu Hà Lan và các điểm khớp lệnh từng phần.

2.4 Đặt cọc và phạt

Cơ chế thứ hai dùng kinh tế để ràng buộc filler. Khi đăng ký nhận một phần lệnh, filler phải nộp một khoản cọc được tính theo cỡ lệnh và thời gian còn lại tới hạn. Nếu filler giao đúng phần đã cam kết, cọc được hoàn lại toàn bộ; trường hợp giao thiếu, phần thiếu bị giữ lại làm khoản phạt.

Thiết kế phân biệt hai trường hợp để nhằm với nhau. Filler “thua cuộc” (do filler khác đã khớp hết phần đó trước) không có lỗi và được hoàn cọc đầy đủ; filler “bỏ dở” — tức là không thực hiện phần đã nhận dù vẫn có thể khớp được — bị phạt mất một phần cọc. Hình 2.2 minh họa cách phân biệt này. Nếu không có cơ chế cọc và phạt, một filler có thể đăng ký nhận lệnh nhưng không thực hiện, khiến lệnh của người dùng bị đình trệ cho đến khi hết hạn.



Hình 2.2: Cách phân biệt hoàn cọc (filler không có lỗi) hay phạt cọc (filler bỏ dở) sau thời hạn.

2.5 Cơ chế dự phòng

Cơ chế thứ ba giúp lệnh không bị bỏ dở khi hết hạn. Khi một lệnh đã gần tới hạn mà vẫn còn phần chưa được khớp, một thành phần dự phòng sẽ tự động hoán đổi nốt phần còn lại thông qua các bộ tổng hợp nhiều sàn (DEX aggregator) để tìm giá tốt nhất có thể từ thanh khoản trên chuỗi — vì khi không còn filler nào sẵn sàng cung cấp nguồn ngoài chuỗi tốt hơn thì đây là phương án cuối còn lại. Điều kiện luôn được giữ là kết quả nhận được phải đạt mức tối thiểu mà người dùng đã ký, và cùng một phần khối lượng không thể bị xử lý hai lần.

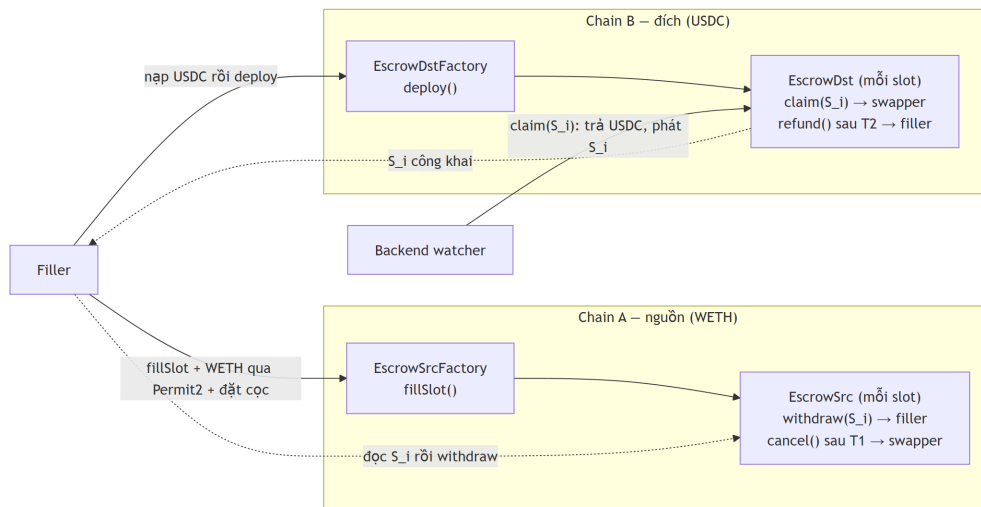
Về khả năng mở rộng, hợp đồng dự phòng có một danh sách trắng (allowlist) các sàn được phép gọi tới, nên không bị khóa cứng vào một sàn duy nhất. Tuy nhiên đây cũng là một giới hạn về mặt thực thi: mỗi aggregator có API và định dạng dữ liệu riêng, vốn không đồng bộ với nhau, nên mỗi sàn mới cần thêm một bộ chuyển đổi (adapter) riêng ở ngoài chuỗi. Trong phạm vi đề án, hệ thống đã triển khai bộ chuyển đổi cho hai sàn là Uniswap và KyberSwap, và có thể thêm các sàn khác như 1inch hay 0x theo cùng cách. Sơ đồ chi tiết của luồng dự phòng được trình bày ở

mục 3.5.

2.6 Giao dịch xuyên chuỗi không sử dụng cầu nối

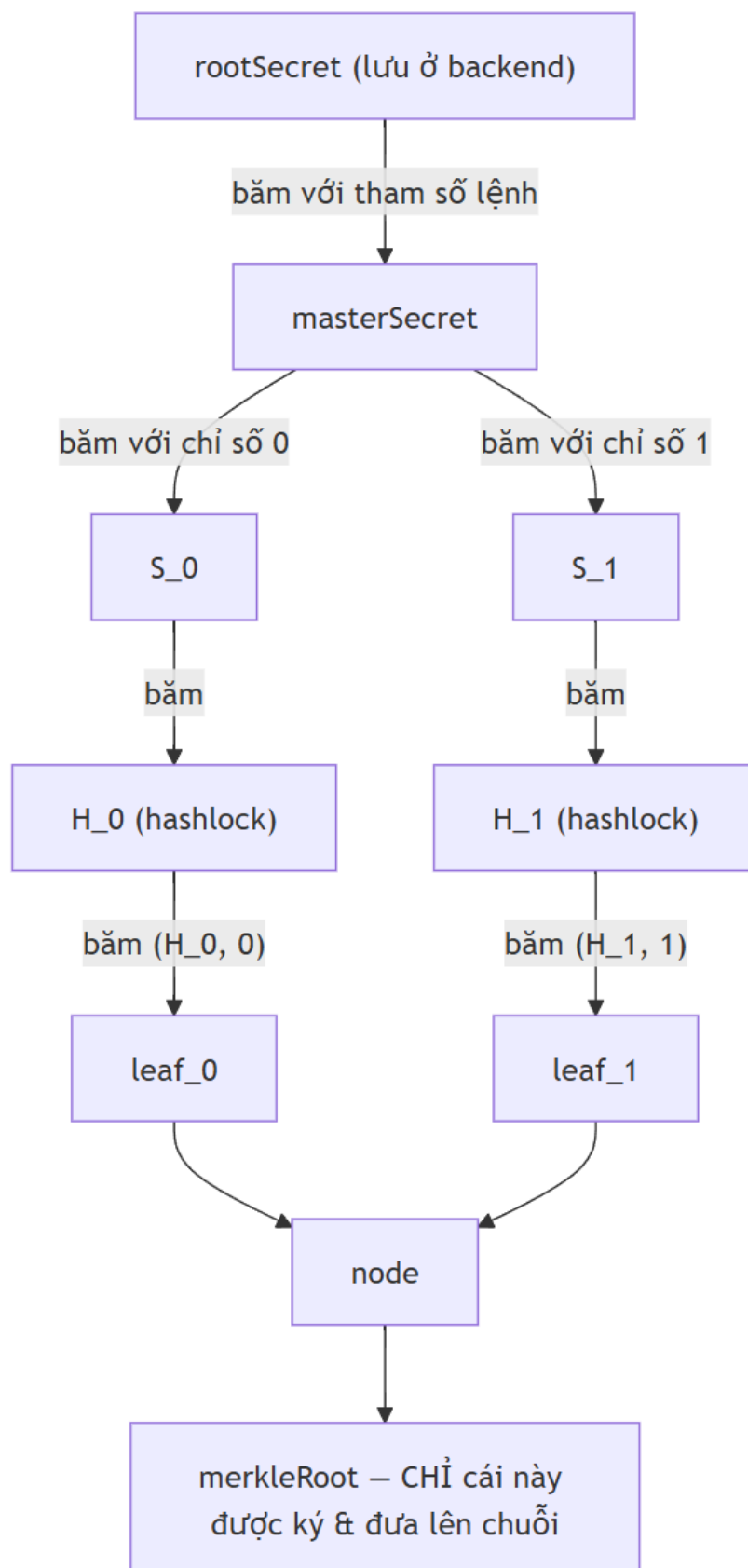
Cơ chế thứ tư mở rộng hệ thống sang giao dịch giữa hai chuỗi. Các giải pháp xuyên chuỗi dựa trên hợp đồng khóa thời gian theo mã băm (Hashed Timelock Contract - HTLC) [3] thường khớp trọn vẹn cả lệnh theo nguyên tắc toàn bộ hoặc không có gì (khái niệm HTLC xem Phụ lục D.4). NeutronX chia một lệnh xuyên chuỗi thành N phần (gọi là slot), mỗi slot có một hợp đồng ký quỹ riêng trên cả hai chuỗi. Kiến trúc này cho phép nhiều filler khớp các slot độc lập với nhau; lỗi tại một slot không lan sang slot khác; tài sản của các slot chưa được nhận không rời khỏi ví người dùng.

Hình 2.3 mô tả kiến trúc trên hai chuỗi: ở mỗi chuỗi có một hợp đồng nhà máy (factory) đảm nhận việc tạo ra các hợp đồng ký quỹ cho từng slot; một thành phần theo dõi (watcher) ở backend đảm nhận việc tiết lộ bí mật để hoàn tất giao dịch. Một điều kiện về thời gian luôn được giữ để bảo vệ filler, đó là thời hạn trên chuỗi đích (T2) luôn sớm hơn thời hạn trên chuỗi nguồn (T1).



Hình 2.3: Kiến trúc giao dịch xuyên chuỗi với một hợp đồng ký quỹ riêng cho mỗi slot trên cả hai chuỗi.

Để khớp được từng phần mà ký một lần duy nhất, hệ thống dùng một cây Merkle các bí mật. Từ một bí mật gốc, hệ thống tính ra bí mật riêng cho từng slot, rồi băm thành các khóa (hashlock) làm lá của cây Merkle; duy nhất giá trị gốc (root) của cây được người dùng và cosigner ký rồi đưa lên chuỗi. Hình 2.4 minh họa cách sắp xếp này. Nhờ vậy, hệ thống cấp được quyền khớp cho từng slot một cách độc lập trong khi cả lệnh dùng một chữ ký duy nhất.



Hình 2.4: Cách tạo bí mật cho từng slot từ một bí mật gốc và cây Merkle tương ứng.

2.7 Đánh đổi trong thiết kế

Mỗi cơ chế trên kéo theo một đánh đổi thiết kế. Việc dùng cosigner đơn giản hóa thiết kế nhưng tạo ra một điểm phụ thuộc tập trung. Việc tính cọc qua TWAP với cửa sổ ngắn phù hợp với môi trường giả lập nhưng dễ bị thao túng hơn cửa sổ dài dùng trong thực tế. Cơ chế dự phòng yêu cầu một bộ chuyển đổi riêng cho mỗi sàn tích hợp. Việc quy giá trị lệnh về ETH đòi hỏi token giao dịch phải có hồ thanh khoản ghép cặp với WETH. Các đánh đổi này và các rủi ro liên quan được phân tích chi tiết ở Chương 4 và Chương 5.

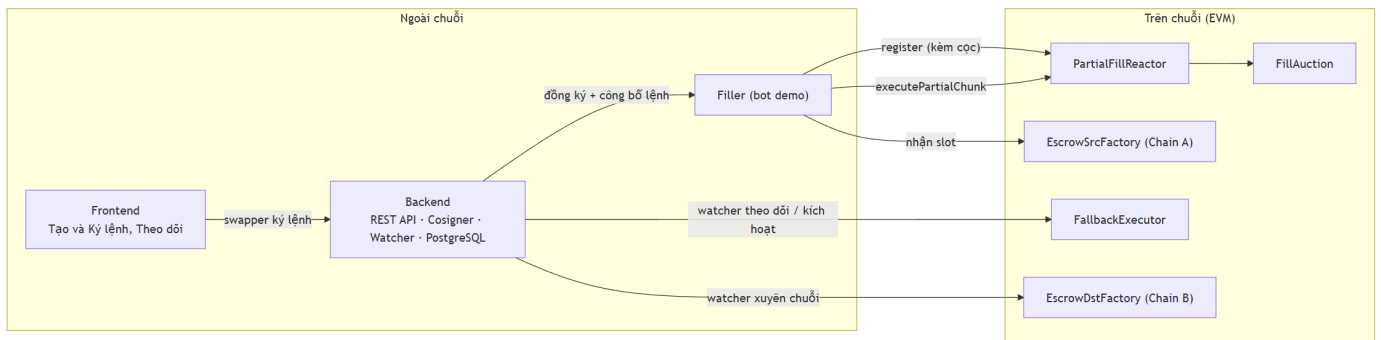
Tóm lại, chương này đã tìm hiểu quá trình phát triển của Uniswap qua ba giai đoạn, từ đó làm rõ những hạn chế mà các hệ thống hiện tại chưa giải quyết được. Dựa trên đó, đề án đề xuất bốn cơ chế: khớp lệnh từng phần với giá được ký, ràng buộc filler bằng cọc và phạt, cơ chế dự phòng qua DEX aggregator, và giao dịch xuyên chuỗi không cầu nối. Chương tiếp theo sẽ đi vào thiết kế và cài đặt chi tiết các cơ chế này ở tầng hợp đồng thông minh.

CHƯƠNG 3. XÂY DỰNG CÁC HỢP ĐỒNG THÔNG MINH

Chương này trình bày phần lõi của đề án: xây dựng các hợp đồng thông minh. Mở đầu là kiến trúc tổng thể, sơ đồ gói và sơ đồ phụ thuộc giữa các hợp đồng, sau đó đi vào từng hợp đồng theo đúng bốn cơ chế đã nêu ở Chương 2. Phần backend và frontend đóng vai trò hỗ trợ và được trình bày ngắn gọn ở cuối chương.

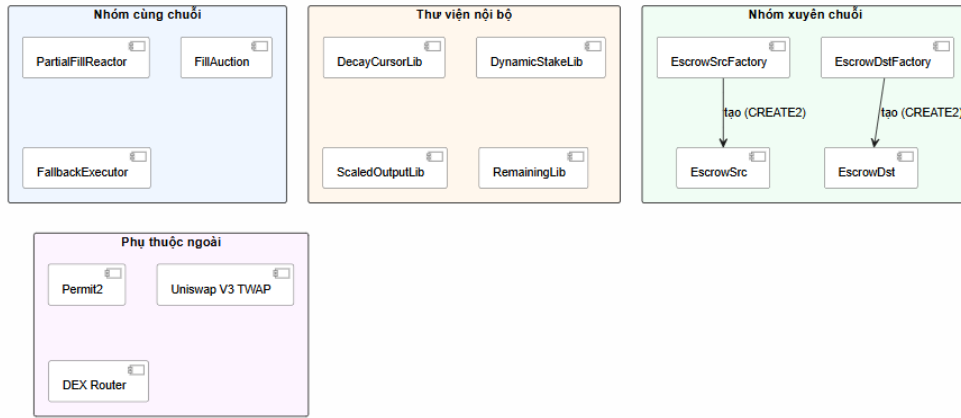
3.1 Kiến trúc tổng thể và luồng giao dịch

Hệ thống NeutronX gồm bốn tầng. Tầng **hợp đồng thông minh** chạy trên chuỗi, chứa toàn bộ logic kiểm tra và thực hiện lệnh; đây là phần quan trọng nhất. Tầng **backend** chạy ngoài chuỗi, đảm nhận việc đồng ký lệnh, lưu trữ và theo dõi trạng thái. Tầng **filler** là các chương trình thực hiện khớp lệnh. Cuối cùng là **giao diện người dùng** để tạo và theo dõi lệnh. Hình 3.1 mô tả cách bốn tầng này nối với nhau.

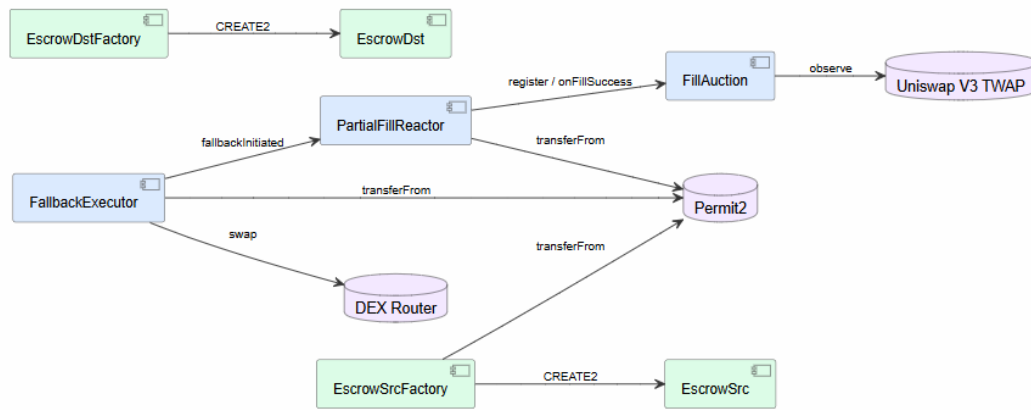


Hình 3.1: Kiến trúc tổng thể của hệ thống NeutronX gồm bốn tầng.

Tầng hợp đồng gồm hai nhóm chính: nhóm **cùng chuỗi** (PartialFillReactor, FillAuction, FallbackExecutor) xử lý lệnh trên một chuỗi duy nhất; nhóm **xuyên chuỗi** (EscrowSrcFactory, EscrowDstFactory và các hợp đồng ký quỹ per-slot) xử lý lệnh giữa hai chuỗi khác nhau. Cả hai nhóm dùng chung một tập thư viện nội bộ và phụ thuộc vào Permit2 để chuyển tài sản. Hình 3.2 phân nhóm các hợp đồng và thư viện, Hình 3.3 thể hiện quan hệ gọi giữa chúng.



Hình 3.2: Sơ đồ gói các hợp đồng thông minh của NeutronX.



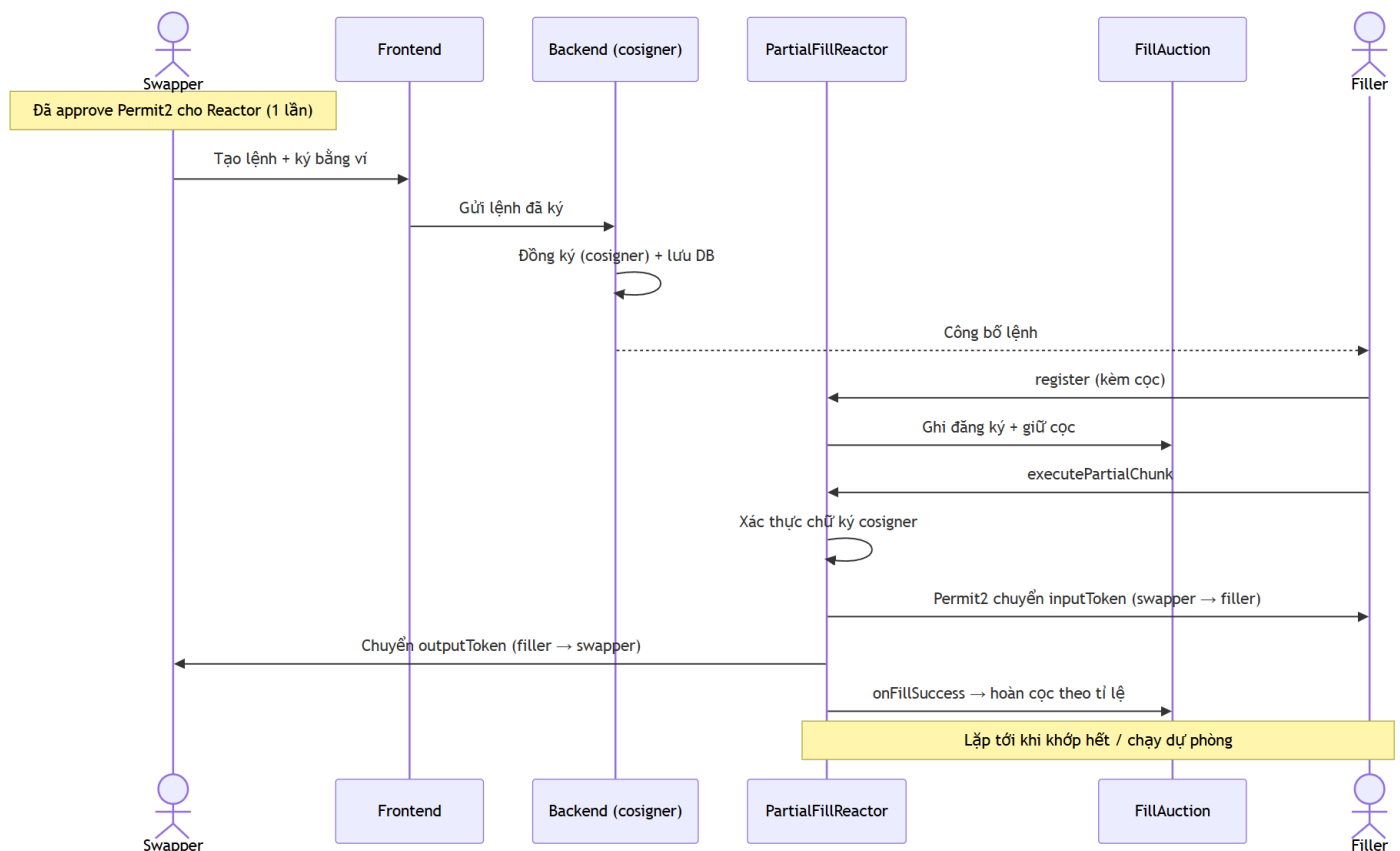
Hình 3.3: Sơ đồ phụ thuộc giữa các hợp đồng.

Vòng đời của một lệnh trong cùng một chuỗi diễn ra qua các bước sau:

1. Người dùng tạo lệnh trên giao diện và ký lệnh bằng ví theo chuẩn EIP-712. Việc ký này diễn ra ngoài chuỗi nên không tốn phí gas.
2. Backend nhận lệnh, đồng ký (cosign) để xác nhận các tham số giá, rồi lưu vào cơ sở dữ liệu và công bố cho các filler.
3. Một filler thấy lệnh có lợi thì đăng ký nhận một phần khối lượng, kèm theo một khoản cọc, thông qua hợp đồng `FillAuction`.
4. Filler gọi hàm khớp lệnh trên `PartialFillReactor`. Hợp đồng dùng `Permit2` để chuyển tài sản vào của người dùng cho filler, đồng thời filler trả tài sản ra cho người dùng.
5. Sau mỗi lần khớp, `FillAuction` hoàn cọc cho filler theo mức họ đã giao đúng cam kết. Quá trình lặp lại cho tới khi lệnh được khớp hết.

6. Nếu gần tới hạn mà lệnh vẫn còn dư, `FallbackExecutor` sẽ khớp nốt phần còn lại.

Hình 3.4 biểu diễn luồng này dưới dạng sơ đồ trình tự.



Hình 3.4: Sơ đồ trình tự vòng đời một lệnh được khớp từng phần trong cùng một chuỗi.

3.2 Mô hình dữ liệu lệnh

Mọi thông tin của một lệnh được gói trong một cấu trúc gọi là `PartialFillOrder`, gồm 11 trường như ở Bảng 3.1. Toàn bộ 11 trường này đều nằm trong phần dữ liệu được ký, nên không bên nào có thể tự ý thay đổi mà không bị phát hiện.

Bảng 3.1: Các trường của một lệnh `PartialFillOrder`.

Trường	Ý nghĩa
<code>swapper</code>	Địa chỉ người tạo lệnh.
<code>inputToken</code>	Token người dùng bán ra.
<code>inputAmount</code>	Tổng khối lượng token bán.
<code>outputToken</code>	Token người dùng muốn nhận.
<code>minOutputAmount</code>	Mức nhận tối thiểu chấp nhận được (chống trượt giá).
<code>deadline</code>	Số block hạn chót của lệnh.
<code>nonce</code>	Số chống lặp lại lệnh.
<code>minFillBps</code>	Tỉ lệ tối thiểu cho mỗi lần khớp (tránh khớp quá vụn).
<code>startPrice</code>	Giá khởi điểm của đường cong đầu giá Hà Lan.
<code>decayPerBlock</code>	Mức giảm giá sau mỗi block.
<code>feeTier</code>	Mức phí pool dùng để tham chiếu giá.

Ba trường cuối (`startPrice`, `decayPerBlock`, `feeTier`) mô tả đường giảm giá. Ba trường này nằm trong phần dữ liệu được ký, do đó đường giảm giá được cosigner xác thực và filler không thể tự đặt tham số giá có lợi cho bản thân.

3.3 Hợp đồng `PartialFillReactor`

`PartialFillReactor` là hợp đồng đảm nhận việc khớp lệnh từng phần. Bảng 3.2 liệt kê các hàm chính.

Bảng 3.2: Các hàm chính của `PartialFillReactor`.

Hàm	Tham số / Kiểu trả về	Mục đích
<code>register</code>	<code>order</code> , <code>sig</code> , <code>cosigSig</code> , <code>fillBps: uint16</code>	Filler cam kết khớp <code>fillBps%</code> lệnh, đặt cọc qua <code>FillAuction</code> .
<code>executePartialChunk</code>	<code>order</code> , <code>sig</code> , <code>cosigSig</code> , <code>fillAmount: uint256</code>	Một lần khớp: Permit2 kéo input từ người dùng, filler trả output.
<code>cancel</code>	<code>order</code>	Người dùng hủy lệnh chưa khớp hết.
<code>invalidateNonce</code>	<code>nonce: uint256</code>	Vô hiệu nonce, hủy mọi lệnh dùng nonce đó.

Hàm `register` kiểm tra chữ ký của lệnh rồi chuyển tiếp yêu cầu đặt cọc sang `FillAuction`. Hàm `executePartialChunk` thực hiện một lần khớp theo trình tự “kiểm tra - thay đổi trạng thái - tương tác” (Checks-Effects-Interactions), một quy ước phổ biến để tránh lỗi gọi lại (reentrancy). Cụ thể, trước khi chuyển tài sản, hợp đồng kiểm tra: lệnh chưa bị hủy, chưa bị chuyển sang chế độ dự phòng,

nonce chưa bị vô hiệu, filler có đăng ký hợp lệ, và lượng khớp không vượt phần còn lại. Sau khi qua hết các kiểm tra, hợp đồng mới cập nhật phần còn lại rồi mới chuyển tài sản.

Mỗi lần khớp được tính giá độc lập tại block đó theo công thức: số tiền ra bằng lượng khớp nhân với giá tại block đó. Cách này tránh được lỗi mà thiết kế ban đầu gặp phải: tích lũy kết quả toàn bộ lệnh có thể gây tràn số ở lần khớp cuối, khiến lệnh bị kẹt. Ngoài ra, mỗi lần khớp phải đạt mức ra tối thiểu theo tỉ lệ; khi lệnh khớp xong, tổng số tiền ra phải đạt `minOutputAmount` đã ký.

Giá hiện tại của lệnh tại block b được tính tuyến tính từ giá khởi điểm P_0 (`startPrice`) và mức giảm mỗi block d (`decayPerBlock`), sà ở 0:

$$P(b) = \max(0, P_0 - (b - b_0) \cdot d) \quad (3.1)$$

trong đó b_0 là block tại lần khởi tạo gần nhất. Tính an toàn của giá dựa trên hai yếu tố: một là biến thời gian b là `block.number` do chuỗi cung cấp, một đại lượng tăng đều mà mọi nút đều xác minh được, nên không bên nào có thể chèn giá trị giả; hai là ba tham số P_0 , d , b_0 đều nằm trong phần dữ liệu được ký và được cosigner xác thực, nên filler không thể tự đặt giá khởi điểm hay tốc độ giảm giá có lợi cho bản thân. Thứ duy nhất filler còn chọn được là thời điểm khớp; khớp muộn nhận được giá thấp hơn nhưng `minOutputAmount` đặt mức sàn từ dưới, do đó khoản giá trị mà filler khớp muộn có thể trích xuất bị giới hạn và không thể khiến người dùng nhận dưới mức đã ký.

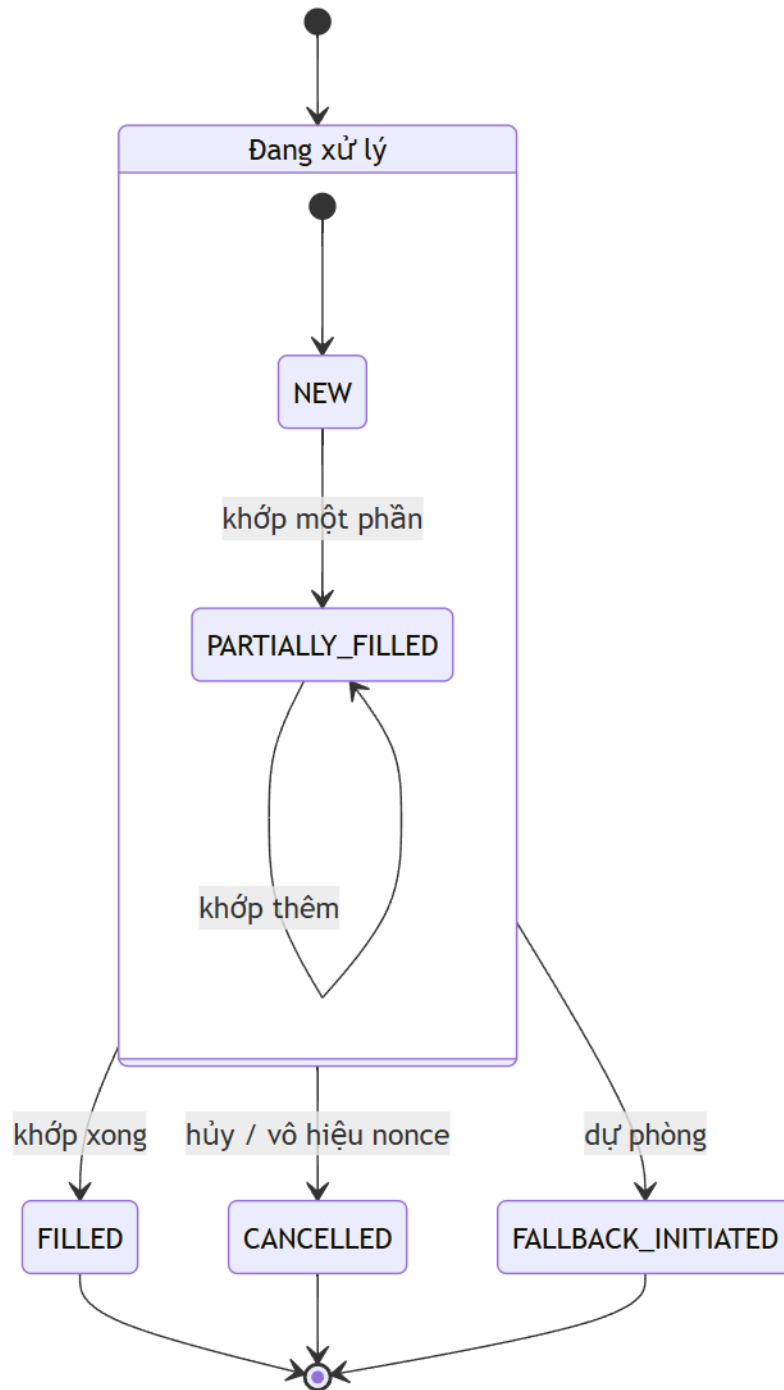
```

1 function getCurrentPrice(DecayCursor storage c) internal view
   returns (uint128) {
2     uint256 blocksPassed = block.number - c.lastResetBlock;
3     uint256 decayed      = blocksPassed * c.decayPerBlock;
4     if (decayed >= c.currentStartPrice) return 0;    // chạm sàn
5     return c.currentStartPrice - uint128(decayed);
6 }

```

Listing 3.1: Tính giá hiện tại theo block (DecayCursorLib).

Một lệnh trong hợp đồng có thể ở các trạng thái như ở Hình 3.5: mới tạo, đang khớp dở, đã khớp xong, hoặc bị hủy / chuyển sang dự phòng. Người dùng cũng có thể chủ động hủy lệnh hoặc vô hiệu một nonce để dừng các lần khớp tiếp theo.



Hình 3.5: Các trạng thái của một lệnh trong hợp đồng PartialFillReactor.

3.4 Hợp đồng FillAuction

FillAuction đảm nhận phần đặt cọc và phạt. Bảng 3.3 liệt kê các hàm chính.

Bảng 3.3: Các hàm chính của FillAuction.

Hàm	Tham số / Kiểu trả về	Mục đích
register	orderHash, filler, fillBps, order- SizeEth, deadline	Ghi đăng ký, tính và khóa cọc theo cỡ lệnh và thời gian còn lại.
onFillSuccess	orderHash, filler, filledBps: uint16	Hoàn cọc theo tỉ lệ lượng thực giao / lượng đã cam kết.
slash	orderHash, filler	Phạt filler bỏ dở sau hạn: 90% vào quỹ, 10% cho người gọi.
release Registration	orderHash, filler	Hoàn 100% cọc khi filler không có lỗi (lệnh đã hủy, nonce vô hiệu...).
withdraw	to: address payable	Filler rút ETH đã được ghi nợ (pull payment).

Khi một filler đăng ký, hợp đồng tính ra số cọc cần nộp và ghi lại đăng ký đó. Số cọc được tính bằng cách quy giá trị phần lệnh về ETH, rồi nhân với một tỉ lệ theo cỡ lệnh và một hệ số theo thời gian còn lại tới hạn (đăng ký càng sát hạn thì hệ số càng cao). Giá trị quy về ETH được lấy từ giá trung bình theo thời gian (TWAP) của pool Uniswap tương ứng.

Cụ thể, số cọc bằng giá trị phần lệnh quy về ETH (V_{eth}) nhân với một tỉ lệ theo cỡ lệnh và một hệ số theo thời gian còn lại:

$$\text{collateral} = V_{eth} \cdot \frac{r_s}{10^4} \cdot \frac{m_t}{10^4} \quad (3.2)$$

Hệ số thời gian m_t tăng dần khi đăng ký càng sát hạn, để chặn việc đăng ký sát hạn với cọc rẻ: $m_t = 1,0$ khi còn > 50 block; $1,5$ khi còn > 20 ; $3,0$ khi còn > 5 ; và $5,0$ khi đã rất sát hạn. Vì cọc tỉ lệ *thăng* với V_{eth} (không có chiều tỉ lệ khớp), một cam kết phần lớn không bao giờ rẻ hơn phần nhỏ — loại bỏ khả năng khai gian cỡ lệnh.

```

1 function computeCollateral(uint256 notionalEth, uint256
   deadline, uint32[4] storage rate)
2   internal view returns (uint256)
3   {
4     uint8 sBucket = getOrderSizeBucketETH(notionalEth); // <1
        / <10 / <100 / >=100 ETH
5     uint8 tBucket = getTimeBucket(deadline); //
        theo so block con lai

```

```

6      uint256 base    = notionalEth * rate[sBucket] / 10000; // x
      tỉ lệ cơ lệnh
7      return base * _getTimeMultiplier(tBucket) / 10000;      // x
      hệ số thời gian
8  }

```

Listing 3.2: Tính cọc động theo cơ lệnh và thời gian (DynamicStakeLib).

Khi filler khớp xong, hàm `onFillSuccess` hoàn cọc theo bảng hoàn cọc ở Bảng 3.4. Mức hoàn được tính theo tỉ lệ giữa lượng filler đã giao và lượng đã *cam kết* của chính filler đó, không phải so với cả lệnh. Nhờ vậy, một filler giao đúng phần đã nhận (dù lớn hay nhỏ) được hoàn lại toàn bộ cọc; khi giao thiếu so với cam kết, phần thiếu bị giữ lại làm khoản phạt.

Bảng 3.4: Bảng hoàn cọc mặc định (đơn vị: phần vạn của số cọc), theo cơ lệnh và tỉ lệ giao so với cam kết.

Cơ lệnh \ Tỉ lệ giao	0–2%	2–10%	10–30%	30–70%	≥70%
Nhỏ (< 1 ETH)	500	1000	2500	5000	10000
Vừa (1–10)	333	1000	2000	5000	10000
Lớn (10–100)	100	333	1000	3333	10000
Rất lớn (> 100)	100	200	667	2000	10000

Phần xử lý sau khi hết hạn được thiết kế để công bằng với filler. Sau thời hạn cộng thêm một khoảng chờ (50 block), nếu một đăng ký vẫn chưa tắt toán thì có hai khả năng. Nếu lệnh đã được khớp xong, đã bị hủy, hoặc nonce đã bị vô hiệu, thì filler không có lỗi và được hoàn lại toàn bộ cọc qua hàm `releaseRegistration`. Ngược lại, nếu lệnh vẫn còn phần khớp được mà filler bỏ dở, thì filler bị phạt qua hàm `slash`: phần lớn cọc (90%) vào quỹ chung, phần còn lại (10%) cho người gọi hàm. Cách phân biệt này đã được minh họa ở Hình 2.2 của Chương 2. Toàn bộ việc rút tiền dùng cơ chế “kéo” (pull payment): tiền được ghi nợ trước, người nhận tự gọi rút sau, để tránh lỗi khi chuyển tiền.

Bảng hoàn cọc được ghi nhận tại thời điểm đăng ký, nên mọi sửa đổi sau đó không ảnh hưởng tới các filler đã đăng ký trước; số cọc tỉ lệ tuyến tính với khối lượng phần cam kết, loại bỏ khả năng cọc cho phần lớn thấp hơn cọc cho phần nhỏ.

3.5 Hợp đồng FallbackExecutor

`FallbackExecutor` đảm nhận cơ chế dự phòng. Bảng 3.5 liệt kê hàm duy nhất của hợp đồng.

Bảng 3.5: Hàm chính của `FallbackExecutor`.

Hàm	Tham số / Kiểu trả về	Mục đích
<code>executeFallback</code>	<code>order, sig, cosigSig, router: address, routerData: bytes, minOut: uint256</code>	Đánh dấu lệnh sang chế độ dự phòng, <code>Permit2</code> kéo phần dư, gọi router trong danh sách trắng.

Cửa sổ hoạt động của hợp đồng giới hạn trong 10 block trước hạn của lệnh. Khi chạy, hợp đồng xác thực chữ ký của lệnh, rồi bật cờ dự phòng cho lệnh và đưa `remaining` về 0 — cả hai thao tác hoàn thành trước khi gọi sang sàn ngoài. Sau đó, mọi lần khớp thường đều bị chặn bởi một trong hai điều kiện độc lập: cờ dự phòng đã bật, hoặc `remaining` đã bằng 0. Phần khối lượng có thể bị xử lý hai lần ở đây chính là phần tài sản vào chưa khớp của người dùng; nếu vừa bị sàn dự phòng đem đi đổi vừa bị một filler khớp thường rút thêm thì tài sản đó sẽ bị kéo hai lần. Vì cả hai luồng cùng thao tác trên `remaining`, và biến này được cập nhật *trước* khi tài sản di chuyển, không còn khe gọi lại (reentrancy) để chen vào — phần khối lượng chưa khớp chỉ có thể do đúng một luồng xử lý.

Sau đó, hợp đồng dùng `Permit2` lấy phần còn lại của người dùng, rồi gọi tới một sàn giao dịch để hoán đổi. Kết quả nhận được phải đạt cả mức tối thiểu mà người dùng đã ký lần mức tối thiểu do route ước tính; nếu thiếu thì cả giao dịch bị hủy. Nếu route không dùng hết tài sản vào, phần thừa được trả lại cho người dùng.

Về các sàn được hỗ trợ, hợp đồng giữ một danh sách trắng các địa chỉ sàn được phép gọi tới. Đồ án đã cài đặt bộ chuyển đổi cho **Uniswap** và **KyberSwap**: bộ chuyển đổi này nằm ở backend, đảm nhận việc gọi API của từng sàn để lấy đường đi tốt nhất rồi mã hóa dữ liệu giao dịch đúng định dạng của sàn đó. Việc bổ sung một sàn mới đòi hỏi viết thêm một bộ chuyển đổi tương tự và thêm địa chỉ sàn vào danh sách trắng, không cần sửa lại hợp đồng.

3.6 Các hợp đồng cho giao dịch xuyên chuỗi

Phần xuyên chuỗi dùng một mô hình khác với hợp đồng khóa thời gian dùng chung thường thấy: mỗi phần (slot) của lệnh có một hợp đồng ký quỹ riêng trên cả hai chuỗi, được tạo ra bằng kỹ thuật `CREATE2`. Trên chuỗi nguồn (ví dụ chuỗi giữ `WETH`), `EscrowSrcFactory` đảm nhận việc tạo các hợp đồng ký quỹ `EscrowSrc`; trên chuỗi đích (ví dụ chuỗi giữ `USDC`) là `EscrowDstFactory` tạo ra các `EscrowDst`. Bảng 3.6 liệt kê các hàm chính của cả bốn hợp đồng.

Bảng 3.6: Các hàm chính của nhóm hợp đồng xuyên chuỗi.

Hàm	Tham số / Kiểu trả về	Mục đích
EscrowSrcFactory		
fillSlot	info, swapperSig, cosignerSig, slotIndex: uint8, hashlock: bytes32, proof: bytes32[] kèm msg.value (cọc) → address	Kiểm 2 chữ ký (lần đầu mỗi lệnh) và Merkle proof của slot, tạo EscrowSrc bằng CREATE2, Permit2 kéo tài sản slot vào.
reopenSlot	orderHash, slotIndex	Mở lại slot cho filler mới sau khi filler cũ bỏ.
EscrowSrc (mỗi slot một instance)		
withdraw	secret: bytes32	Filler rút tài sản nguồn khi tiết lộ đúng secret.
cancel	—	Sau T1: hoàn tài sản cho người dùng, tịch thu cọc an toàn của filler.
EscrowDstFactory		
deploy	hashlock: bytes32, filler, token, amount, deadline → address	Filler tạo EscrowDst bằng CREATE2, khóa tài sản đích vào hợp đồng mới.
EscrowDst (mỗi slot một instance)		
claim	secret: bytes32	Backend/watcher giải phóng tài sản đích khi có secret.
refund	—	Sau T2: hoàn tài sản về cho filler nếu chưa bị claim.

Khi một filler nhận một slot, hợp đồng nhà máy kiểm tra hai chữ ký (của người dùng và của cosigner) cùng với bằng chứng Merkle cho slot đó, rồi mới tạo hợp đồng ký quỹ và dùng Permit2 chuyển đúng phần tài sản của slot vào hợp đồng vừa tạo. Filler phải nộp khoản đặt cọc an toàn, khiến hành vi chiếm giữ slot mà không thực hiện trở nên tốn kém. Cây Merkle (mục 2.6) cho phép xác thực từng slot độc lập với chỉ một chữ ký cho toàn lệnh.

Cụ thể, từ khóa chính M được tính ngoài chuỗi từ khóa gốc và dữ liệu lệnh, mỗi slot i có khóa (S_i), hashlock (h_i) và lá Merkle riêng:

$$\begin{aligned}
 S_i &= H(M \parallel i) && \text{(khóa của slot } i) \\
 h_i &= H(S_i) && \text{(hashlock)} \\
 \text{leaf}_i &= H(H(h_i \parallel i)) && \text{(lá Merkle, băm hai lần)}
 \end{aligned} \tag{3.3}$$

với H là keccak256. Lá được băm *hai lần* theo chuẩn của thư viện OpenZepelin [4] để chống tấn công tiền ảnh thứ hai (xem Phụ lục D.3). Chỉ gốc Merkle (merkleRoot) được người dùng và cosigner ký rồi đưa lên chuỗi; các S_i không được lưu trữ mà được tính lại khi cần. Khi một filler nhận slot, hợp đồng xác minh cặp (h_i, i) là một lá hợp lệ của cây *trước khi* tiếp cận tài sản, ngăn filler tự chọn hashlock mà bản thân biết trước secret tương ứng.

```

1 bytes32 leaf = keccak256(bytes.concat(keccak256(abi.encode(
    hashlock, slotIndex))));
2 require(MerkleProof.verify(merkleProof, order.merkleRoot, leaf)
    , "invalid merkle proof");
3 // ... chỉ sau dòng này mới Permit2 kéo tài sản của slot vào
    escrow

```

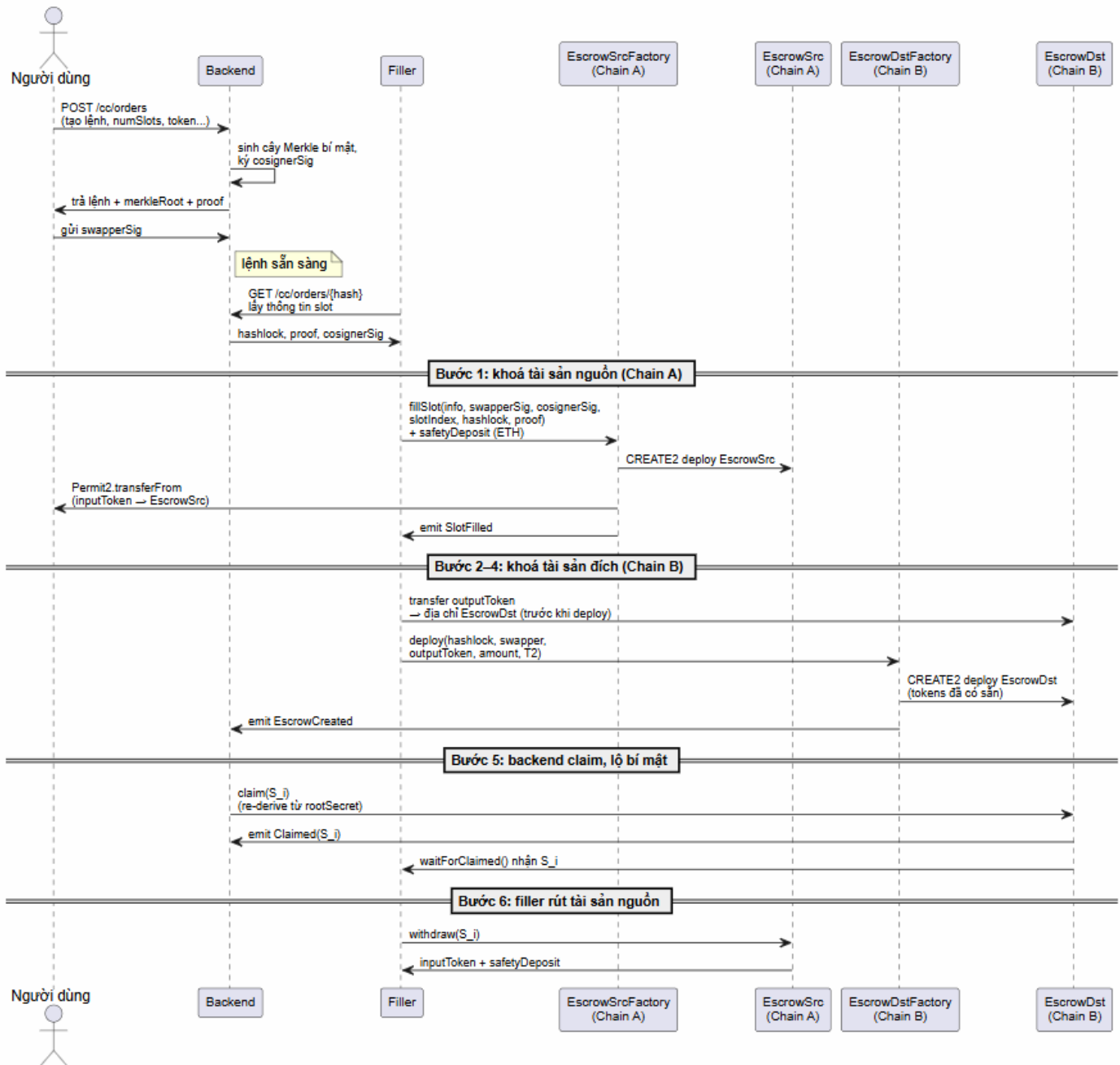
Listing 3.3: Xác minh lá Merkle của slot trước khi kéo tài sản (EscrowSrcFactory).

Bảng 3.7 so sánh mô hình này với cách dùng một hợp đồng khóa thời gian dùng chung.

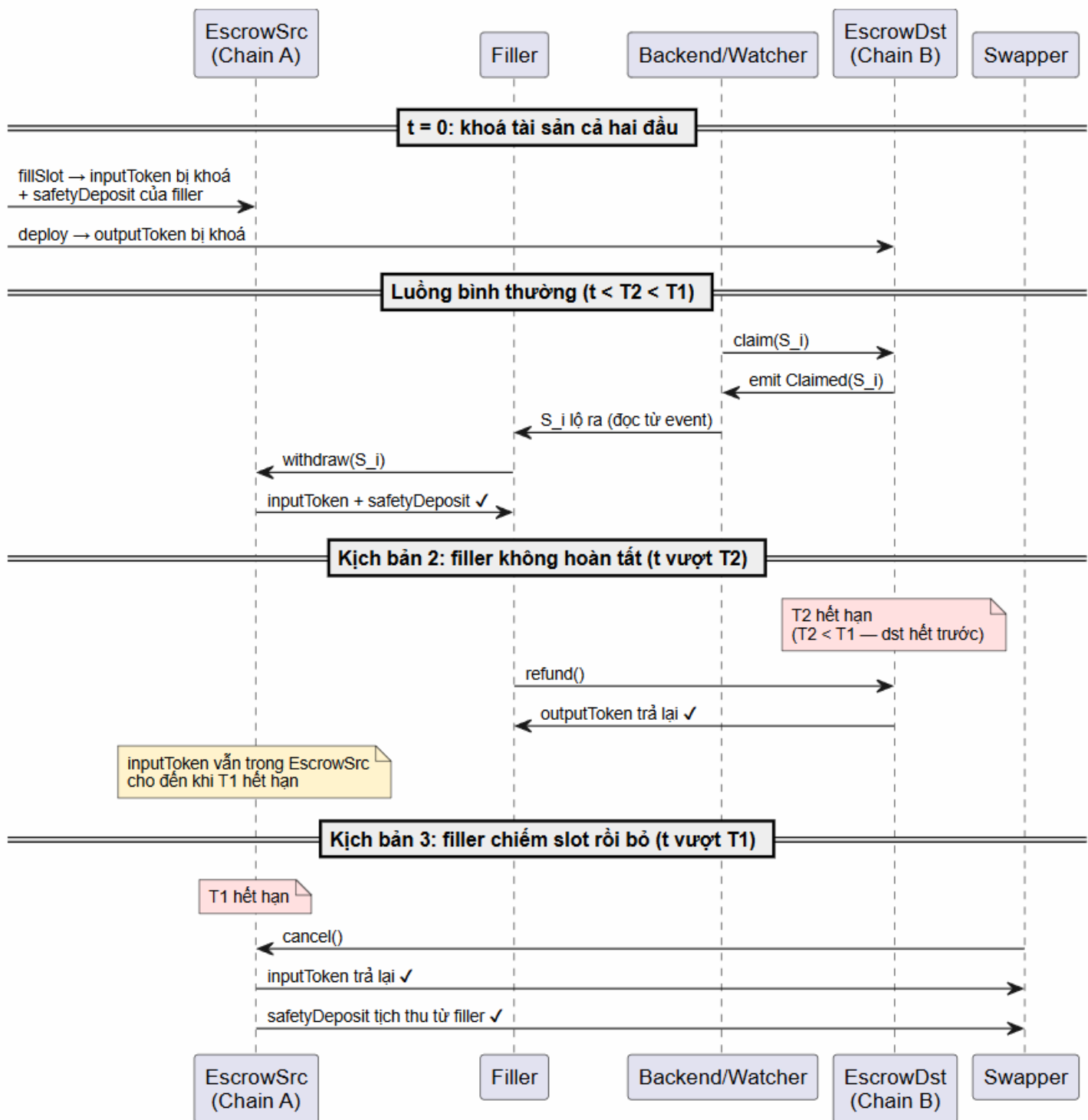
Bảng 3.7: So sánh hai cách làm xuyên chuỗi.

Tiêu chí	Hợp đồng dùng chung	Hợp đồng riêng mỗi slot
Cô lập sự cố	Một lỗi ảnh hưởng mọi filler	Một hợp đồng hỏng không ảnh hưởng slot khác
Tài sản của slot chưa khớp	Nằm trong hợp đồng từ đầu	Vẫn nằm trong ví người dùng
Đặt cọc an toàn của filler	Không có	Có, mỗi slot một khoản
Dễ kiểm tra trạng thái	Khó	Mỗi hợp đồng tự báo trạng thái riêng

Một điều kiện thời gian luôn được giữ để bảo vệ filler: hạn trên chuỗi đích (T2) luôn sớm hơn hạn trên chuỗi nguồn (T1). Nhờ vậy, một filler đã bỏ tiền ra trên chuỗi đích nhưng vì lý do nào đó không hoàn tất được thì vẫn lấy lại được tiền của mình trước khi phần tài sản trên chuỗi nguồn bị hoàn cho người dùng. Hình 3.6 và Hình 3.7 minh họa luồng 6 bước và ràng buộc thời gian T1/T2.



Hình 3.6: Sơ đồ trình tự luồng giao dịch xuyên chuỗi (6 bước).



Hình 3.7: Ràng buộc thời gian $T2 < T1$ bảo vệ filler khỏi mất cả hai đầu.

3.7 Backend và Frontend

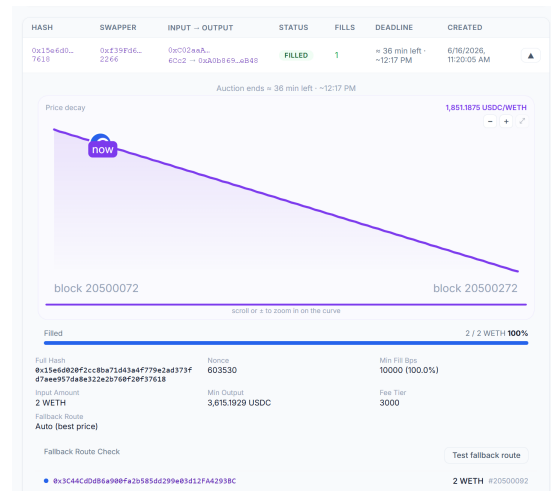
Ngoài tầng hợp đồng, hệ thống còn có backend và frontend để kết nối người dùng với các hợp đồng. **Backend** viết bằng Node.js và TypeScript, dùng cơ sở dữ liệu PostgreSQL. Backend cung cấp các API để tạo và đọc lệnh, giữ vai trò *cosigner* (đồng ký để xác nhận đường giảm giá và gốc cây Merkle), và chạy các thành phần

CHƯƠNG 3. XÂY DỰNG CÁC HỢP ĐỒNG THÔNG MINH

theo dõi (watcher) để tự động kích hoạt cơ chế dự phòng cũng như hoàn tất giao dịch xuyên chuỗi. Backend không giữ tài sản của người dùng: tài sản luôn nằm trong các hợp đồng trên chuỗi, và khi backend gặp sự cố, tiền của người dùng vẫn an toàn. **Frontend** là giao diện web để người dùng tạo và ký lệnh, theo dõi tiến độ khớp, và xem đường giảm giá; Hình 3.8 minh họa một số màn hình chính.

The screenshot shows a 'Swap' interface. At the top, 'You sell' is set to 2 WETH with a balance of 48. Below, 'You receive (market estimate)' is 3651.71 USDC with a balance of 103,724.7442. The 'Auction range' is 3,724.7442 to 3,615.1929 USDC. A 'Price decay' slider is set to 0.5326. The 'Fallback route' is 'Uniswap (Smart Order Router)'. The 'Min fill' is 100%. The 'Fillable by fillers' bar is at 100%. The 'Price (start - floor)' is 1810.71 to 1757.45 USDC/WETH. A large purple 'Swap' button is at the bottom.

(a) Màn hình tạo lệnh.



(b) Màn hình theo dõi lệnh và đồ thị đầu giá.

Hình 3.8: Một số màn hình của giao diện người dùng.

The screenshot shows a 'Swap' interface with 'Chain A' and 'Chain B' both set to 31337. The 'You send' is 2 WETH. The 'You receive (minimum)' is 2 WETH. The 'Deadline (Chain A - 31337 block)' is 20500272. The 'T2 buffer (blocks)' is 50. The 'Nonce' is 402438. A large purple 'Swap' button is at the bottom.

(a) Màn hình tạo lệnh xuyên chuỗi.

2 WETH → 2 WETH		ACTIVE
7/18 slots filled - deadline = 1d 7h left - T2 = 1d 7h left		
Order	0x1720e17ae05534816ab426ab6997a26cdcf41d80bade0c0d5b3c159f	copy
▲ hide slot hashes		
Slot 0	Hashlock: 0x291722f230a1a1e4949910137999b3a083038422649a4a7220f2f2d113a Escrow: 0x1820b0c0b0a133a7f19a07350b1a4a449947c	DONE copy
Slot 1	Hashlock: 0x43e9f4f20939f5d46477c524509a212779a4a32059da2a3c3d70c334390e Escrow: 0x40d1c99fca709a23b5553a0201a0c4f0d1c99e	DONE copy
Slot 2	Hashlock: 0x2761a5d8427290c1a1045904f58d28205a9b77c313e14e3245f9a4b002f Escrow: 0x70b3ab289a2a01c498534346840f221388a9e	DONE copy
Slot 3	Hashlock: 0x43e9f4f20939f5d46477c524509a212779a4a32059da2a3c3d70c334390e Escrow: 0x43e9f4f20939f5d46477c524509a212779a4a32059da2a3c3d70c334390e	DONE copy
Slot 4	Hashlock: 0x43e9f4f20939f5d46477c524509a212779a4a32059da2a3c3d70c334390e Escrow: 0x43e9f4f20939f5d46477c524509a212779a4a32059da2a3c3d70c334390e	AVAILABLE copy
Slot 5	Hashlock: 0x43e9f4f20939f5d46477c524509a212779a4a32059da2a3c3d70c334390e Escrow: 0x43e9f4f20939f5d46477c524509a212779a4a32059da2a3c3d70c334390e	AVAILABLE copy

(b) Màn hình theo dõi trạng thái từng slot.

Hình 3.9: Giao diện người dùng cho giao dịch xuyên chuỗi.

Việc các hợp đồng và cơ chế vừa trình bày có hoạt động đúng như thiết kế hay không sẽ được kiểm chứng qua các thử nghiệm ở chương tiếp theo.

CHƯƠNG 4. KIỂM THỬ VÀ RÀ SOÁT BẢO MẬT

Với một hệ thống giữ tài sản của người dùng, kiểm thử không dừng ở mức “chạy được” — cần chứng minh được rằng các tính chất an toàn giữ nguyên ngay cả khi bị tấn công. Chương này trình bày cách đồ án tổ chức kiểm thử theo từng tính chất cụ thể, rồi tóm tắt kết quả rà soát bảo mật độc lập từ Trufy [5].

4.1 Phương pháp và môi trường kiểm thử

Bộ kiểm thử dùng ba kiểu bổ trợ nhau, tất cả viết bằng Foundry:

- **Kiểm thử đơn vị:** dựng từng tình huống cụ thể rồi kiểm kết quả.
- **Kiểm thử sinh ngẫu nhiên (fuzz):** công cụ tự sinh hàng nghìn bộ dữ liệu đầu vào và kiểm một tính chất phải luôn đúng, nên kết luận rộng hơn vài ví dụ tay.
- **Kiểm thử bất biến (invariant):** sinh chuỗi thao tác ngẫu nhiên và sau mỗi bước kiểm một tính chất quan trọng không được vi phạm.

Môi trường chạy là mạng giả lập Anvil: một mạng fork từ Ethereum mainnet (có sẵn token và pool Uniswap thật để tính cọc theo giá), và hai mạng riêng cho kịch bản xuyên chuỗi. Một lưu ý quan trọng: filler trong kiểm thử được cấp vốn lớn và không tốn phí gas thật, nên kết quả chứng minh *tính đúng của các luồng và cơ chế*, không phản ánh cạnh tranh kinh tế khi vốn hữu hạn. Khi chạy lại toàn bộ bằng `forge test`, cả 149 trường hợp kiểm thử trên 24 bộ test đều đạt; kết quả tổng hợp theo thành phần như Bảng 4.1.

Bảng 4.1: Tổng hợp kết quả `forge test`: 149/149 trường hợp đạt, nhóm theo thành phần.

Nhóm thành phần	Số test đạt
PartialFillReactor (khớp từng phần, định giá, sản output)	38
FillAuction (cọc, phạt, hoàn cọc, bất biến solvency)	35
Chống trích xuất giá trị bằng MEV	11
FallbackExecutor (cơ chế dự phòng)	9
Hợp đồng xuyên chuỗi (escrow theo slot, ràng buộc thời gian)	13
Thư viện tính giá/cọc và oracle TWAP	43
Tổng cộng	149

4.2 Các tính chất đã chứng minh

Chương 3 xây dựng bốn cơ chế: khớp lệnh từng phần, đặt cọc và phạt filler, dự phòng tự động, và giao dịch xuyên chuỗi. Mỗi cơ chế đi kèm các kịch bản lỗi hoặc tấn công tiềm ẩn; sáu tính chất dưới đây kiểm tra từng kịch bản đó:

1. **Tính đúng và không bị kẹt của khớp từng phần** — tích lũy kết quả qua nhiều nhịp có thể gây tràn số hoặc để lại phần dư không tắt toán được.
2. **Bảo đảm mức ra tối thiểu cho người dùng** — filler kiểm soát lượng trả ra và có thể trả thiếu hoặc thao túng tham số giá.
3. **Cơ chế cọc và phạt kháng khai thác kinh tế** — filler có thể khai báo sai phần đăng ký hoặc bỏ dở để giảm mức phạt.
4. **MEV nằm trong ngưỡng chấp nhận của lệnh** — bot có thể trì hoãn để khớp ở mức giá thấp hơn, trích xuất giá trị trong biên độ giảm giá.
5. **Cơ chế dự phòng loại trừ khớp trùng và bảo đảm hoàn tất** — nếu dự phòng và filler thường xử lý cùng một phần khối lượng, tài sản bị rút hai lần.
6. **Giao dịch xuyên chuỗi kháng griefing và sai lệch thời gian** — filler có thể chiếm slot mà không thực hiện, hoặc vi phạm ràng buộc $T_2 < T_1$ khiến filler mất tài sản trên cả hai chuỗi.

Mỗi mục dưới đây nêu tính chất và các kiểm thử tương ứng.

4.2.1 Khớp lệnh từng phần luôn đúng và không bị kẹt

Nhóm này kiểm hợp đồng `PartialFillReactor`. Hai tính chất cần bảo đảm: lệnh luôn có thể khớp hết và tấn công gọi lại trong lúc trả tiền bị chặn.

Lệnh không bị kẹt. Thiết kế ban đầu tích lũy kết quả toàn lệnh, gây tràn số ở nhịp cuối và khiến lệnh không tắt toán được; thiết kế hiện tại định giá độc lập từng nhịp, loại bỏ lỗi này. Kiểm thử `fuzz testFuzz_alwaysSettleable` sinh ngẫu nhiên lịch giảm giá và cách chia nhịp, xác nhận lệnh khớp được về 0 với mọi đầu vào.

`MinFillRemainder.t.sol` kiểm tra trường hợp phần còn lại nhỏ hơn ngưỡng tối thiểu: với `minFillBps = 50%` và tổng 100 đơn vị, filler thứ nhất khớp 60, để lại 40 đơn vị. Kiểm thử xác nhận hợp đồng cho phép filler thứ hai khớp toàn bộ 40 đơn vị còn lại, đảm bảo lệnh có thể tắt toán.

Chống tấn công gọi lại (reentrancy). Kiểm thử dùng một token ERC-20 mô phỏng tấn công: khi hợp đồng gọi token để chuyển tài sản ra, token gọi ngược lại hàm khớp trước khi trạng thái được cập nhật. Kiểm thử xác nhận lần gọi thứ hai bị từ chối do `remaining` đã được ghi về 0 trước khi thực hiện bất kỳ lần chuyển tài

sản nào.

```

1 // Token doc: khi duoc goi de tra tien, no nhay nguoc vao hop
  dong thu rut them.
2 function transferFrom(address from, address to, uint256 amount)
3     public override returns (bool)
4 {
5     if (armed) {
6         armed = false;                // chi re-enter dung
          mot lan
7         (bool ok, bytes memory ret) = target.call(
          attackCalldata);
8         if (!ok) { assembly { revert(add(ret, 0x20), mload(ret)
          ) } }
9     }
10    return super.transferFrom(from, to, amount);
11 }
    
```

Kiểm thử `test_reentrantOutputToken_reverts` xác nhận khóa reentrancy phát hiện gọi lồng nhau và revert toàn bộ giao dịch ngay trước khi tiền rời khỏi hợp đồng.

4.2.2 Người dùng không bao giờ bị trả thiếu

Nhóm này xác minh hợp đồng thực thi đúng giá và mức sàn được ký trong lệnh.

Giả mạo và khớp dưới sàn. Kiểm thử `test_tamperStartPrice_rejected` xác nhận nếu filler cố sửa giá khởi điểm trong lệnh thì chữ ký không khớp và giao dịch bị từ chối. Kiểm thử `test_fillBelowMinOutput_reverts` xác nhận một nhịp khớp trả ra dưới mức tối thiểu theo tỉ lệ cũng bị chặn.

Bẫy làm tròn xuống. Do phép chia lấy phần nguyên, tổng các mức sàn từng nhịp có thể nhỏ hơn mức sàn tuyệt đối người dùng đã ký. Kiểm thử `test_completionBelowAbsoluteFloor_reverts` dựng ví dụ cụ thể: lệnh bán 3 đơn vị, người dùng ký yêu cầu nhận tối thiểu 5 USDC tổng cộng.

- Nhịp A: filler khớp 2 đơn vị $\Rightarrow$ sàn nhịp A = $\lfloor 5 \times 2/3 \rfloor = 3$ USDC, trả 3 ✓
- Nhịp B: filler khớp 1 đơn vị $\Rightarrow$ sàn nhịp B = $\lfloor 5 \times 1/3 \rfloor = 1$ USDC, trả 1 ✓
- Tổng nhận: $3 + 1 = 4$ USDC < 5 USDC mà người dùng đã ký $\Rightarrow$ nhịp hoàn tất bị chặn

Hai nhịp đều qua sàn riêng, nhưng cộng lại vẫn thiếu vì phần lẻ của phép chia bị bỏ mỗi lần. Hợp đồng có chốt chặn thứ hai ở nhịp hoàn tất: tổng đã trả phải $\geq$ mức sàn tuyệt đối, nếu không thì revert với lỗi “min output total”. Kiểm thử `fuzz_testFuzz_completedOrder_neverBelowSignedFloor` xác nhận tính chất

này đúng với nhiều cách chia lệnh khác nhau.

4.2.3 Cơ chế cọc và phạt không bị khai thác kinh tế

Nhóm này kiểm tra năm tình huống khai thác cơ chế cọc và xác nhận mỗi tình huống được chặn đúng.

Giả mạo khi đăng ký. Khi đăng ký, filler phải nộp cọc tỉ lệ với phần lệnh cam kết. Nếu hợp đồng chấp nhận con số do filler tự khai thay vì đọc từ lệnh đã ký, filler có thể khai nhận 1% nhưng thực khớp 100%, khiến cọc không đủ để phạt khi bỏ dở. Kiểm thử `RegistrationForgery` xác nhận hợp đồng đọc phần trăm trực tiếp từ lệnh đã ký, loại bỏ khả năng này.

Bị filler khác giành phần. Tình huống: filler A đăng ký 100% và nộp cọc đủ; filler B khớp trước 60%, để lại 40% cho filler A. Kiểm thử `test_frontRunRace_loserReclaimsFullStake` xác nhận filler A không bị phạt: hợp đồng tính lại theo tỉ lệ thực giao (40%), hoàn đúng 40% cọc và không báo lỗi.

Không bị phạt oan trong các trường hợp khác. Hai tình huống còn lại cũng được xác nhận là filler không bị phạt và lấy lại đủ cọc: khi người dùng chủ động hủy lệnh, và khi nonce của lệnh bị vô hiệu. Tên các hàm kiểm thử tương ứng (nhóm `test_H*`) được liệt kê ở Phụ lục C.

Khớp phần nhỏ và chặn khớp vụn. `test_snipeSmallChunk_fullyRefunded` xác nhận filler khớp phần nhỏ nhưng đúng cam kết vẫn được hoàn đủ cọc. `test_minFillBps_blocksDustFill` chặn việc khớp quá vụn dưới ngưỡng tối thiểu.

Hợp đồng luôn đủ tiền chi trả. Kiểm thử bất biến `invariant_solveny` duy trì một biến ghost theo dõi tổng cọc kỳ vọng: cộng khi có đăng ký, trừ khi tất toán hoặc phạt. Sau mỗi thao tác ngẫu nhiên, số dư thực tế của hợp đồng phải bằng biến ghost; bất kỳ sai lệch nào đều khiến kiểm thử thất bại.

```

1 // "so sach bong" chay song song:
2 function register(...) { ghost_activeStake += stake; } // + coc
3 function fill(...)      { ghost_activeStake -= stake; } // - coc
   khi xong
4 function slash(...)     { ghost_activeStake -= stake; } // - coc
   khi phạt
5
6 // sau MOI thao tac ngau nhien, so that phai khop so bong:
7 function invariant_solveny() public view {
8     assertEq(address(auction).balance, ghost_activeStake +
        totalPending);
9 }

```

Kịch bản tổng hợp. Kiểm thử `test_population_honestAndMev_invariantsHold` chạy ba lệnh với nhiều người tham gia:

- **Lệnh 1:** filler trung thực đăng ký và khớp đủ 100% $\Rightarrow$ hoàn toàn bộ cọc.
- **Lệnh 2:** filler trung thực đăng ký nhưng bot MEV giành mất toàn bộ. Sau deadline, filler trung thực rút lại đủ cọc, không bị phạt.
- **Lệnh 3:** bot MEV đăng ký rồi bỏ dở, bị phạt mất cọc. Filler trung thực khớp thay.

Sau ba lệnh, kiểm thử kiểm một loạt tính chất: người dùng nhận đủ hoặc trên sàn, tổng token vào ra cân bằng, filler trung thực không mất tiền, hợp đồng vẫn đủ khả năng chi trả.

4.2.4 Chống trích xuất giá trị bằng MEV

Tính chất này liên quan tới cả `PartialFillReactor` (định giá) và `FillAuction` (phạt). Kiểm thử `test_lateFill_skimsDecaySpread_butStaysAboveFloor` mô phỏng bot MEV và đo lường giá trị trích xuất được.

Bối cảnh: lệnh bán 4 WETH, giá khởi điểm 2 500 USDC/WETH giảm dần theo từng block, mức nhận tối thiểu cả lệnh là 9 000 USDC.

- **Block sớm — filler trung thực:** khớp 2 WETH ở giá cao, trả người dùng 5 000 USDC.
- **40 block sau — bot MEV:** chờ giá giảm xuống khoảng 2 300 USDC/WETH rồi mới khớp 2 WETH còn lại. Bot phải trả ít nhất theo sàn pro-rata: $9\,000 \times 2/4 = 4\,500$ USDC, nhưng trả ít hơn 5 000 USDC so với filler trung thực.

Ba dòng kiểm tra với số cụ thể:

```
1 assertLt(mevPaid, 5_000e6); // bot tra ít hơn
   filler trung thực
2 assertGe(mevPaid, 4_500e6); // nhưng phan này vẫn
   >= san pro-rata
3 assertGe(usdc.balanceOf(swapper), 9_000e6); // tổng người dùng
   nhận >= san cả lệnh
```

Bot trích xuất được phần chênh giữa giá thị trường lúc nó khớp và mức sàn — khoảng vài trăm USDC. Nếu bot chờ thêm đến khi giá rơi dưới 4 500 USDC thì giao dịch bị từ chối, như kiểm thử `test_lateFill_belowFloor_reverts` xác nhận. Kiểm thử `test_registerThenAbandon_isSlashed_noProfit` kiểm thêm: đăng ký rồi bỏ dở không giúp bot thu lợi mà còn khiến bot bị phạt mất cọc.

Hệ thống không loại bỏ hoàn toàn MEV — bot vẫn kiếm được một ít trong

khoảng người dùng đã chấp nhận. Đây là giới hạn thiết kế được bàn thêm ở mục 4.3.

4.2.5 Ngăn khớp trùng và bảo đảm hoàn tất lệnh

Nhóm này kiểm `FallbackExecutor` — hợp đồng tự thực hiện hoán đổi thay khi không có filler nào khớp trước hạn.

`test_fallback_swapsSuccessfully_viaRouter` xác nhận luồng chính: khi gần hết hạn mà lệnh còn dư, hợp đồng gọi một sàn trong danh sách trắng để hoán đổi phần còn lại. Các điều kiện biên được kiểm tra và chặn đầy đủ: filler thường không thể khớp sau khi dự phòng đã chạy (ngăn khớp trùng); gọi tới sàn ngoài danh sách trắng, chạy sai thời điểm, hoặc trả ra dưới mức người dùng đã ký đều bị từ chối; tài sản đầu vào không dùng hết được hoàn lại cho người dùng. Tên các hàm kiểm thử tương ứng được liệt kê ở Phụ lục C.

4.2.6 Xuyên chuỗi: chống griefing và an toàn về thời gian

Nhóm này kiểm các hợp đồng ký quỹ xuyên chuỗi. Luồng chính chạy được qua `test_fillSlot_then_withdraw_paysFiller`. Các điều kiện biên đều bị chặn sớm trước khi đựng tài sản: bằng chứng Merkle sai (`test_fillSlot_invalidProof_reverts`), chữ ký người dùng sai (`test_fillSlot_invalidSwapperSig_reverts`), slot bị khớp lần thứ hai (`test_fillSlot_secondTime_revertsSlotAlreadyFilled`), cọc an toàn bằng 0 (`test_fillSlot_zeroSafetyDeposit_reverts`).

Hết hạn mà chưa hoàn tất. Kiểm thử `test_cancel_afterExpiry_refundsSwapper_andPaysSwapperDeposit` kiểm tình huống:

1. Người dùng gửi tài sản vào hợp đồng nguồn (chain A), chờ filler khóa tài sản tương ứng ở chain B.
2. Filler không hoàn thành trong thời hạn.
3. Sau khi hết hạn, người dùng gọi `cancel`: nhận lại tài sản nguồn *cộng thêm* cọc an toàn của filler bị tịch thu. Người dùng không mất tiền và được bồi thường nhẹ cho thời gian bị chờ.

Thứ tự thời gian $T2 < T1$. Ràng buộc $T2$ (hạn hợp đồng đích) $< T1$ (hạn hợp đồng nguồn) đảm bảo filler đủ thời gian hoàn tất trên chuỗi nguồn sau khi tiết lộ `secret` trên chuỗi đích. Kiểm thử `test_T2geqT1_swapperTakesBothLegs` dựng kịch bản vi phạm $T2 = T1$: filler tiết lộ `secret` trên chuỗi đích để nhận tài sản, nhưng khi dùng `secret` đó trên chuỗi nguồn thì $T1$ đã qua — người dùng hủy hợp đồng nguồn lấy lại tài sản, đồng thời dùng `secret` vừa lộ để rút tài sản đích, khiến filler mất tài sản trên cả hai chuỗi. Hợp đồng ngăn tình huống này bằng cách từ chối mọi lệnh có $T2 \geq T1$ tại thời điểm tạo.

Toàn bộ còn được chạy đầu cuối với hai filler khớp hai slot khác nhau của cùng một lệnh, kết thúc bằng việc kiểm lại số dư của tất cả các bên trên cả hai chain.

4.3 Độ phủ và phạm vi kiểm thử

Tổng cộng 149 trường hợp kiểm thử trên 24 bộ test đều đạt khi chạy lại bằng `forge test` (Bảng 4.1). Độ phủ dòng lệnh đo bằng `forge coverage: FallbackExecutor 100% (41/41)`, `FillAuction 98% (112/114)`, `PartialFillReactor 95% (93/98)`; các thư viện `DecayCursorLib` và `ScaledOutputLib` đạt 100%. Nhóm hợp đồng xuyên chuỗi có độ phủ thấp hơn (`EscrowSrcFactory 97%`, các hợp đồng ký quỹ theo slot 74–83%), do một số nhánh hoàn tiền và hủy yêu cầu điều kiện môi trường thực để kích hoạt. Danh sách đầy đủ được liệt kê ở Phụ lục C.

Bộ kiểm thử không bao phủ một số điều kiện thực tế. Anvil fork không có mempool thật và filler được cấp vốn không giới hạn, nên kết quả mới chứng minh các cơ chế đúng, chưa phản ánh cạnh tranh kinh tế khi nhiều filler vốn hữu hạn cùng tham gia trên mainnet. Một số rủi ro bảo mật như thao túng TWAP hay grieflock xuyên chuỗi cũng cần vốn thật và môi trường production mới khai thác được — đây là những thứ nằm ngoài tầm của local fork và được bàn kỹ hơn ở mục 4.4. Backend và frontend được chạy thử thủ công để minh họa luồng; tối ưu gas không phải trọng tâm nên không được đo hệ thống.

4.4 Rà soát bảo mật bởi Trufy

Ngoài bộ kiểm thử tự viết, đồ án sử dụng Trufy¹ để audit độc lập toàn bộ mã nguồn hợp đồng. Báo cáo đầy đủ được đính kèm tại Phụ lục E [5].

4.4.1 Tổng quan phát hiện

Trufy ghi nhận 8 phát hiện: 3 mức cao, 4 mức trung bình, 1 mức thấp. Bảng 4.2 liệt kê tất cả kèm trạng thái xử lý.

¹<https://trufy.io> — công cụ rà soát bảo mật hợp đồng thông minh tự động bằng AI, không phải kiểm tra bởi chuyên gia người.

Mục	Tên phát hiện (rút gọn)	Mức	Xử lý
3.1	Cross-chain grief-lock: cọc an toàn cố định	Cao	Đã giải trình
3.2	Token fee-on-transfer qua kế toán danh nghĩa	Cao	Đã sửa
3.3	Watcher bỏ qua sự kiện lịch sử sau outage	Cao	Đã sửa
3.4	Nonce xuyên chuỗi không kiểm tra on-chain	Trung bình	Đã giải trình
3.5	TWAP đơn pool 60 giây dễ bị thao túng	Trung bình	Đã giải trình
3.6	Slot được mở lại không thể thử lại bởi cùng filler	Trung bình	Đã giải trình
3.7	<code>FillAuction.withdraw()</code> kẹt ETH với non-payable	Trung bình	Đã sửa
3.8	Fallback watcher hỗ trợ cố định 6 token	Thấp	Ngoài phạm vi

Bảng 4.2: Tám phát hiện từ báo cáo Trufy và trạng thái xử lý.

4.4.2 Phát hiện đã khắc phục

3.2 — Token fee-on-transfer (Cao). Hợp đồng cũ kiểm tra mức sàn output dựa trên *số danh nghĩa* mà nó yêu cầu filler chuyển, chứ không phải số người dùng thực nhận; một token tự trừ phí khi chuyển sẽ làm người dùng nhận dưới sàn mà giao dịch vẫn lọt.

Sửa: trong `executePartialChunk`, hợp đồng nay đo *chênh lệch số dư thực tế* của người dùng quanh lần chuyển (`balanceOf` trước và sau), rồi áp cả mức sàn lẫn kế toán lên số thực nhận đó. Kiểm thử với một token thu phí 2% xác nhận một lần khớp khiến người dùng nhận dưới sàn nay bị revert thay vì âm thâm trả thiếu (`test_feeOnTransferOutput_belowFloor_reverts`), và khi vẫn trên sàn thì người dùng được ghi đúng số thực nhận (`test_feeOnTransferOutput_aboveFloor_creditsActualReceived`).

3.3 — Watcher bỏ qua sự kiện sau outage dài (Cao). Khi backend khởi động lại sau khi dừng lâu, checkpoint của watcher nhảy thẳng về block hiện tại, bỏ qua mọi sự kiện trong khoảng trống.

Sửa: logic checkpoint nay phân biệt hai trường hợp — chuỗi bị đặt lại (checkpoint vượt quá đỉnh hiện tại, buộc lùi về đỉnh) và backlog thật sau outage (giữ checkpoint cũ để quét bù). Phần quét bù được chia thành từng đoạn block có giới hạn để một lần truy vấn log không vượt giới hạn của nhà cung cấp RPC. Việc tái hiện trọn vẹn vẫn cần một lần dừng dài thực tế trên production, nhưng cơ chế bỏ sót đã được loại bỏ ở mức mã nguồn.

3.7 — `FillAuction.withdraw()` kẹt ETH (Trung bình). Filler là smart contract không có hàm `receive()` thì không nhận được ETH push trực tiếp. Hàm `withdraw()` cũ luôn push về `msg.sender` — nếu thất bại thì revert, ETH bị

kết vĩnh viễn ở hợp đồng.

Sửa: hàm `withdraw()` nay nhận thêm tham số `address payable to`, cho phép filler chuyển hướng ETH về bất kỳ địa chỉ payable nào, ví dụ một ví EOA hoặc treasury có `receive()`.

4.4.3 Các phát hiện cần môi trường production để kiểm chứng

Năm phát hiện còn lại không thể tái hiện đầy đủ trên Anvil fork vì chúng đòi hỏi những điều kiện đặc thù của mainnet: vốn lớn để thao túng thị trường, mempool công khai để front-run, hay lượng token đa dạng vượt ngoài 6 token demo.

3.1 — Cross-chain grief-lock (Cao). Về lý thuyết, kẻ xấu thấy tham số `fillSlot` trên mempool có thể giành trước một slot rồi bỏ, khóa tài sản người dùng tới hạn. Mỗi đe dọa này phát sinh trên production: cần mempool công khai để thấy tham số, và cần kinh tế thật để việc grief đáng giá — cả hai đều không tồn tại trên local fork. Hai lớp phòng vệ đã có sẵn: tham số `fill` (chữ ký cosigner, hashlock, Merkle proof) được backend cấp riêng cho filler đã xác thực nên kẻ ngoài không đủ thông tin; và mỗi lần nhận slot bắt buộc phải nộp một khoản cọc an toàn dương (cọc bằng 0 bị từ chối), nên việc chiếm chỗ luôn tốn chi phí thật.

Điều Trufy chỉ ra là khoản cọc này hiện cố định (`MIN_SAFETY_DEPOSIT`), chưa tỉ lệ với giá trị slot. Đây thực chất là *chế độ tắt oracle* của cơ chế cọc: cọc tỉ lệ đòi hỏi phải quy giá trị slot ra ETH bằng một oracle giá (TWAP), mà oracle phát huy tác dụng trên mạng thật. Hệ thống đã có tiền lệ cho đúng cách làm này ở phía cùng chuỗi — `FillAuction` tính cọc động qua TWAP nhưng chạy với oracle tắt (cọc theo lượng danh nghĩa) trong môi trường kiểm thử. Vì vậy, nâng cọc xuyên chuỗi thành tỉ-lệ-giá-trị là việc *cấu hình lúc triển khai* (bật oracle và đặt hệ số), không phải thay đổi logic lõi của escrow; và khi bật, nó đi kèm phần gia cố của sổ TWAP đã nêu ở phát hiện 3.5.

3.4 — Nonce xuyên chuỗi không kiểm tra on-chain (Trung bình). Tính duy nhất của (swapper, nonce) hiện do backend DB đảm bảo. Bảo vệ replay đã có qua `orderHash` duy nhất — hai lệnh nonce giống nhau nhưng tham số khác sẽ hash khác. Thêm nonce registry on-chain là việc cần làm khi mở rộng lên multi-backend production.

3.5 — TWAP đơn pool 60 giây (Trung bình). Kẻ có vốn lớn có thể bơm pool 60 giây để đẩy TWAP về thấp, hạ cọc xuống gần 0. Không thể tái hiện trên local fork vì không có vốn thật. Hợp đồng đã có `minCollateral` làm sàn tuyệt đối, nhưng nâng cửa sổ TWAP và bổ sung oracle độc lập là hướng phát triển rõ ràng.

3.6 — Slot mở lại không thể thử lại bởi cùng filler (Trung bình). Salt CRE-

ATE2 dẫn xuất từ hashlock và địa chỉ filler va chạm với lần deploy cũ, nên filler không thể nhận lại slot mình đã bỏ. Trường hợp này hiếm, xảy ra sau một chuỗi lỗi cụ thể trên production; watcher đã có cơ chế reset slot về trạng thái available.

3.8 — FallbackWatcher hỗ trợ cố định 6 token (Thấp). Watcher hardcode metadata của 6 token mainnet cho mục đích demo. Hỗ trợ token động cần DB token đầy đủ và môi trường production — nằm ngoài phạm vi đề án.

4.5 Kết luận chương

Bộ kiểm thử xác nhận các tính chất an toàn cốt lõi hoạt động đúng trên môi trường giả lập: lệnh không bị kẹt, người dùng không bị trả thiếu, cơ chế cọc không bị khai thác, MEV bị giới hạn trong ngưỡng người dùng chấp nhận, và giao dịch xuyên chuỗi an toàn trước griefing. Audit từ Trufy ghi nhận 8 phát hiện: ba đã sửa, năm còn lại cần điều kiện production để kiểm chứng đầy đủ. Chi tiết ở Phụ lục C và E.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đồ án đã xây dựng được hệ thống NeutronX, một ứng dụng tổng hợp giao dịch phi tập trung theo kiến trúc intent solver, và kiểm chứng nó trên môi trường mạng giả lập. So với năm mục tiêu đặt ra ở mục 1.2.1, kết quả đạt được như Bảng 5.1.

Bảng 5.1: Đối chiếu kết quả với các mục tiêu ban đầu.

Mục tiêu	Kết quả
Khớp lệnh từng phần theo đấu giá Hà Lan	Đã cài đặt trong <code>PartialFillReactor</code> , mỗi phần tính giá độc lập, đã kiểm thử.
Ký lệnh ngoài chuỗi bằng EIP-712 và Permit2	Đã làm; người dùng ký bằng ví, không tốn phí gas khi tạo lệnh.
Đặt cọc và phạt ràng buộc filler	Đã cài đặt trong <code>FillAuction</code> , phân biệt được filler thua cuộc và filler bỏ dở.
Cơ chế dự phòng đảm bảo lệnh hoàn tất	Đã cài đặt trong <code>FallbackExecutor</code> , hỗ trợ Uniswap và KyberSwap.
Giao dịch xuyên chuỗi không cầu nối, khớp từng phần	Đã cài đặt theo mô hình ký quỹ riêng mỗi slot, chạy đầu cuối thành công với hai filler.

Như vậy, cả năm mục tiêu đều đã đạt được ở mức cài đặt và kiểm thử trên môi trường giả lập. Đóng góp đáng kể nhất của đồ án là cơ chế đặt cọc và phạt ràng buộc filler, và phần giao dịch xuyên chuỗi có hỗ trợ khớp từng phần - hai điểm mà các hệ thống đã khảo sát ít làm. Quá trình thực hiện đồ án cho thấy một sai sót nhỏ trong logic hợp đồng thông minh có thể dẫn tới mất tài sản người dùng, từ đó xác nhận tầm quan trọng của kiểm thử kỹ và rà soát bảo mật độc lập.

5.2 Hạn chế

Đồ án có một số hạn chế:

- **Chưa lên mainnet.** Hệ thống hiện được chạy trên mạng giả lập, chưa được triển khai lên mạng thử nghiệm hay mạng chính thức.
- **Chưa mô phỏng cạnh tranh kinh tế thực tế.** Các filler trong kiểm thử được cấp vốn không giới hạn, do đó kết quả kiểm chứng giới hạn ở tính đúng của luồng, chưa phản ánh hành vi khi vốn có hạn; cũng chưa có thanh khoản và người dùng thực.
- **Cosigner tập trung.** Khóa đồng ký (cosigner) hiện do một bên duy nhất nắm

giữ, tạo ra điểm tin tưởng tập trung trong hệ thống.

- **Phạm vi hỗ trợ hẹp.** Phần xuyên chuỗi hiện hỗ trợ các chuỗi tương thích EVM; cơ chế dự phòng hiện tích hợp Uniswap và KyberSwap.

5.3 Hướng phát triển

Từ các hạn chế trên, đồ án có thể được phát triển tiếp theo các hướng sau:

- **Triển khai và kiểm toán.** Đưa hệ thống lên mạng thử nghiệm rồi tiến tới mainnet, kèm theo một đợt kiểm toán bảo mật do chuyên gia thực hiện.
- **Phi tập trung hóa cosigner.** Chạy module backend ký lệnh trong môi trường thực thi tin cậy (TEE — Trusted Execution Environment), để phần ký chạy trong một vùng cô lập không thể can thiệp từ bên ngoài mà không cần tin vào một bên duy nhất.
- **Tăng độ bền cơ chế tính cọc.** Kéo dài cửa sổ lấy giá trung bình (TWAP) và kết hợp thêm nguồn giá độc lập để khó thao túng hơn.
- **Mở rộng dự phòng.** Thêm bộ chuyển đổi cho các sàn khác như 1inch và 0x.
- **Mở rộng xuyên chuỗi.** Hỗ trợ nhiều hơn hai chuỗi EVM, tiến tới các chuỗi không tương thích EVM.

TÀI LIỆU THAM KHẢO

- [1] S. M. Werner, D. Perez, L. Gudgeon, A. Klages-Mundt, D. Harz, and W. J. Knottenbelt, “SoK: Decentralized finance (DeFi),” *Proceedings of the 4th ACM Conference on Advances in Financial Technologies (AFT)*, 2022.
- [2] P. Daian et al., “Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability,” in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 910–927.
- [3] M. Herlihy, “Atomic cross-chain swaps,” in *Proceedings of the 2018 ACM Symposium on Principles of Distributed Computing (PODC)*, 2018, pp. 245–254.
- [4] OpenZeppelin, *OpenZeppelin contracts — thư viện hợp đồng thông minh (MerkleProof, Clones, SafeERC20)*, <https://docs.openzeppelin.com/contracts>, 2024.
- [5] Trufy, “Smart contract security audit report for NeutronX,” Trufy, Security Audit Report, Jun. 2026, Toàn văn báo cáo tại Phụ lục E.
- [6] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” *Ethereum Project Yellow Paper*, 2014.
- [7] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology — CRYPTO ’87*, 1988, pp. 369–378.
- [8] K. Qin, L. Zhou, and A. Gervais, “Quantifying blockchain extractable value: How dark is the forest?” In *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 198–214.
- [9] Ethereum Foundation, *Ethereum developer documentation*, <https://ethereum.org/en/developers/docs/>, 2024.
- [10] Solidity Team, *Solidity language documentation*, <https://docs.soliditylang.org/>, 2024.
- [11] R. Nair, L. Logvinov, and J. Evans, *EIP-712: Typed structured data hashing and signing*, <https://eips.ethereum.org/EIPS/eip-712>, Ethereum Improvement Proposal, 2017.
- [12] M. Lundfall, *EIP-2612: Permit — ERC-20 signed approvals*, <https://eips.ethereum.org/EIPS/eip-2612>, Ethereum Improvement Proposal, 2020.

- [13] Foundry Contributors, *The foundry book*, <https://book.getfoundry.sh/>, 2024.
- [14] Uniswap Labs, *UniswapX protocol documentation*, <https://docs.uniswap.org/contracts/uniswapx/overview>, 2023.
- [15] H. Adams, N. Zinsmeister, M. Salem, R. Keefer, and D. Robinson, *Uniswap v3 core whitepaper*, <https://uniswap.org/whitepaper-v3.pdf>, 2021.
- [16] Uniswap Labs, *Permit2 — next generation token approvals*, <https://github.com/Uniswap/permit2>, 2022.
- [17] Uniswap Labs, *Uniswap V3 oracle — time-weighted average price (TWAP)*, <https://docs.uniswap.org/concepts/protocol/oracle>, 2021.
- [18] KyberNetwork, *KyberSwap developer documentation*, <https://docs.kyberswap.com/>, 2024.
- [19] Gnosis Team, *WETH9 — wrapped ether contract*, <https://etherscan.io/address/0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2>, 2017.
- [20] Circle Internet Financial, *USD Coin (USDC) technical documentation*, <https://www.circle.com/en/usdc>, 2024.
- [21] Tether Operations Limited, *Tether (USDT) technology*, <https://tether.to/en/technology/>, 2024.
- [22] MakerDAO, *DAI stablecoin system*, <https://makerdao.com/en/whitepaper/>, 2020.

PHỤ LỤC

A. MỘT SỐ USE CASE KHÁC

Phụ lục này mô tả một số use case không được trình bày chi tiết trong phần thân, gồm bước cấp quyền chỉ tiêu một lần, bước kiểm tra đăng ký của filler, và hai API truy vấn giá của backend.

A.1 Cấp quyền chỉ tiêu một lần qua Permit2

- **Tác nhân:** Người dùng (swapper).
- **Tiền điều kiện:** Người dùng có token ERC-20 muốn bán và đã kết nối ví.
- **Luồng chính:**
 1. Người dùng gọi `approve` cho hợp đồng Permit2 trên token muốn bán (làm một lần cho mỗi token).
 2. Người dùng cấp hạn mức cho hợp đồng cần dùng (Reactor hoặc EscrowSrcFactory) thông qua Permit2.
- **Hậu điều kiện:** Hợp đồng được phép kéo token của người dùng tới đúng địa chỉ đích khi lệnh được khớp, mà người dùng không phải ký lại hay trả phí gas cho từng lần khớp.

Đây là lý do người dùng “khóa” tài sản mà không cần gửi trước vào một hợp đồng trung gian: tài sản vẫn nằm trong ví cho tới đúng lúc một phần lệnh được khớp.

Ví dụ A.1. Người dùng có 5 000 USDC và muốn đổi dần thành WETH. Họ gọi `usdc.approve(permit2, type(uint256).max)` một lần duy nhất. Sau đó, mỗi lần tạo lệnh 1 000 USDC, họ cấp hạn mức đúng 1 000 USDC cho Reactor qua Permit2. Phần còn lại (4 000 USDC) vẫn nằm trong ví, không bị khóa, và được rút khi từng lệnh tiếp theo được tạo và khớp thành công.

A.2 Kiểm tra đăng ký hợp lệ của filler

- **Tác nhân:** Filler, hợp đồng `PartialFillReactor`.
- **Tiền điều kiện:** Filler đã đăng ký và đặt cọc cho một phần lệnh qua `FillAuction`.
- **Luồng chính:**
 1. Filler gọi hàm khớp lệnh với khối lượng mong muốn.
 2. Hợp đồng kiểm tra qua `hasValidRegistration`: đăng ký đúng là của filler này, chưa tắt toán, chưa bị phạt, khối lượng xin khớp không vượt phần đã cam kết, và chưa quá hạn.

3. Nếu hợp lệ, hợp đồng tiếp tục khớp; nếu không, giao dịch bị từ chối.

- **Hậu điều kiện:** Duy nhất filler có đăng ký hợp lệ được phép khớp lệnh; các bên không đặt cọc không thể tham gia.

Ví dụ A.2. Có một lệnh tổng 10 000 USDC. Filler A đăng ký cam kết khớp 60% (6 000 USDC) và đặt cọc tương ứng. Khi gọi `executePartialChunk` với `fillAmount = 6000 USDC`, hợp đồng xác nhận đúng cam kết và cho phép. Nếu Filler A sau đó thử gọi thêm `fillAmount = 500 USDC` (vượt phần đã đăng ký), giao dịch bị từ chối. Filler B có thể đăng ký phần còn lại (4 000 USDC) một cách độc lập.

A.3 Truy vấn giá thị trường

- **Tác nhân:** Frontend hoặc filler muốn xem giá trước khi tạo lệnh.
- **Tiền điều kiện:** Backend đang chạy và có thể kết nối đến ít nhất một aggregator (Uniswap, KyberSwap...).
- **Tham số:** `inputToken`, `outputToken`, `inputAmount`, `inputDecimals`, `outputDecimals`.
- **Luồng chính:**
 1. Frontend gửi `GET /quote` với thông tin cặp token và khối lượng.
 2. Backend lấy giá từ aggregator tốt nhất hiện có, tính giá thị trường theo TWAP và ước tính số token nhận được.
 3. Backend trả về `estimatedOutput`, `marketRate`, `priceImpact` và nguồn aggregator đã dùng.
- **Hậu điều kiện:** Người dùng thấy được ước tính số token nhận trước khi quyết định tạo lệnh.

Ví dụ A.3. Người dùng muốn biết đổi 1 WETH được bao nhiêu USDC. Frontend gọi `GET /quote` với tham số:

```
{
  "inputToken": "WETH",
  "outputToken": "USDC",
  "inputAmount": "1000000000000000000"
}
```

Backend truy vấn Uniswap và trả về:

```
{
  "estimatedOutput": "3850000000",
  "marketRate": "3850.00",
}
```

```
"priceImpact":      "0.12%",
"source":           "uniswap"
}
```

Người dùng thấy ngay giá ước tính (3 850 USDC) và mức trượt giá trước khi xác nhận.

A.4 Gợi ý tham số lệnh tự động

- **Tác nhân:** Frontend khi người dùng muốn tạo lệnh mà không tự chỉnh tham số đường cong giá.
- **Tiền điều kiện:** Có ít nhất một filler đăng ký phục vụ cặp token tương ứng.
- **Tham số:** `inputToken`, `outputToken`, `inputAmount`; tùy chọn: `deadlineBlocks` (mặc định 100), `targetCoverageBps` (mặc định 9000, tức 90%), `premiumBps` (mặc định 200, tức +2%), `slippageBps` (mặc định 100, tức -1%).
- **Luồng chính:**
 1. Backend lấy giá thị trường hiện tại và số block hiện tại.
 2. Backend tính `startPrice` (giá khởi điểm cao hơn thị trường `premiumBps`), `minOutputAmount` (sàn tuyệt đối thấp hơn thị trường `slippageBps`), và `decayPerBlock` sao cho giá giảm đều từ đỉnh xuống sàn trong `deadlineBlocks` block.
 3. Backend tìm kiếm nhị phân trên danh sách `minFillBps` để chọn giá trị lớn nhất (ít phân mảnh nhất) mà các filler đang đăng ký vẫn đáp ứng được ít nhất `targetCoverageBps%` khối lượng.
 4. Backend trả về toàn bộ tham số sẵn sàng để tạo lệnh: `startPrice`, `decayPerBlock`, `minOutputAmount`, `deadline`, `minFillBps`, kèm ghi chú về mức độ phủ của các filler.
- **Hậu điều kiện:** Người dùng nhận được bộ tham số tối ưu có thể dùng trực tiếp để tạo lệnh mà không cần hiểu chi tiết về đường cong đầu giá Hà Lan.

Ví dụ A.4. Người dùng muốn đổi 1 WETH, thị trường đang ở 3 850 USDC/WETH. Backend tự động tính:

- $startPrice = 3850 \times 1,02 = 3927$ USDC (cao hơn thị trường 2% để hấp dẫn filler nhanh).
- $minOutputAmount = 3850 \times 0,99 = 3811$ USDC (sàn tuyệt đối thấp hơn thị trường 1%).

- $\text{decayPerBlock} = (3927 - 3811) / 100 = 1,16 \text{ USDC/block}$ (giá giảm đều trong 100 block).
- $\text{minFillBps} = 2000 (20\%)$ — giá trị lớn nhất mà các filler đang đăng ký vẫn có thể phủ ít nhất 90% khối lượng.

Người dùng nhận bộ tham số hoàn chỉnh và xác nhận ký lệnh.

B. CƠ SỞ DỮ LIỆU

Backend dùng cơ sở dữ liệu PostgreSQL để lưu lệnh và trạng thái khớp. Phụ lục này liệt kê các bảng chính. Hai bảng `orders` và `fills` phục vụ giao dịch trong cùng một chuỗi; nhóm bảng `cc_*` phục vụ giao dịch xuyên chuỗi. Cơ sở dữ liệu lưu thông tin lệnh và trạng thái, không giữ tài sản của người dùng.

B.1 Bảng `orders` (lệnh trong cùng chuỗi)

Cột	Ý nghĩa
<code>hash</code>	Mã băm của lệnh (khóa chính).
<code>swapper</code>	Địa chỉ người tạo lệnh.
<code>input_token / output_token</code>	Token bán ra / token muốn nhận.
<code>input_amount / min_output</code>	Khối lượng bán / mức nhận tối thiểu.
<code>deadline / nonce</code>	Hạn chót (block) / số chống lặp lệnh.
<code>min_fill_bps</code>	Tỉ lệ tối thiểu cho mỗi lần khớp.
<code>start_price / decay_per_block / fee_tier</code>	Tham số đường cong giá đầu giá Hà Lan.
<code>start_block</code>	Block bắt đầu giảm giá.
<code>signature</code>	Chữ ký của lệnh.
<code>status</code>	Trạng thái lệnh (pending, filled...).
<code>fallback_initiated</code>	Đã chuyển sang chế độ dự phòng hay chưa.
<code>preferred_aggregator</code>	Sàn ưu tiên cho cơ chế dự phòng (rỗng = tự chọn).

B.2 Bảng `fills` (các lần khớp)

Cột	Ý nghĩa
<code>id</code>	Khóa chính (mã giao dịch + chỉ số log).
<code>order_hash</code>	Lệnh tương ứng (khóa ngoại tới <code>orders</code>).
<code>filler</code>	Địa chỉ filler thực hiện lần khớp.
<code>fill_amount / output_amount</code>	Khối lượng khớp / số tiền trả ra.
<code>tx_hash / block_number</code>	Mã giao dịch và số block.
<code>path / graph</code>	Dữ liệu phụ về đường khớp (nếu có).

B.3 Các bảng cho giao dịch xuyên chuỗi

Nhóm bảng `cc_*` lưu trạng thái xuyên chuỗi, gồm: `cc_sessions` lưu bí mật gốc và địa chỉ cosigner cho mỗi người dùng; `cc_orders` lưu thông tin lệnh xuyên

chuỗi (số slot, gốc cây Merkle, hai chữ ký, hạn T2, mã hai chuỗi...); `cc_slots` lưu từng slot (khóa băm, lá Merkle, bằng chứng, trạng thái, filler được giao, địa chỉ hợp đồng ký quỹ đã triển khai); và `cc_fillers` lưu danh sách filler đăng ký phục vụ.

Bảng	Vai trò
<code>cc_sessions</code>	Phiên của người dùng: bí mật gốc, cosigner.
<code>cc_orders</code>	Lệnh xuyên chuỗi: số slot, gốc Merkle, chữ ký, hạn T2.
<code>cc_slots</code>	Từng slot: khóa băm, bằng chứng Merkle, trạng thái, escrow.
<code>cc_fillers</code>	Danh sách filler đăng ký phục vụ.

Ngoài ra còn một số bảng phụ trợ như `tokens` (danh sách token hiển thị) và `indexer_state` (mốc block mà các thành phần theo dõi đã xử lý tới).

C. DANH SÁCH TRƯỜNG HỢP KIỂM THỬ

Phụ lục này liệt kê các trường hợp kiểm thử quan trọng kèm kịch bản tóm tắt, nhóm theo thành phần. Tên hàm giữ nguyên như trong mã nguồn để tiện tra cứu.

Tên hàm test	Kịch bản
PartialFillReactor — khớp lệnh từng phần	
testFuzz_alwaysSettleable	Sinh ngẫu nhiên lịch giảm giá và cách chia lệnh, lệnh luôn khớp được về 0.
test_completionBelowAbsoluteFloor_reverts	Tổng các sàn từng nhip nhỏ hơn sàn tuyệt đối, nhip hoàn tất bị chặn.
testFuzz_completedOrder_neverBelowSignedFloor	Fuzz: tổng tiền ra khi hoàn tất luôn $\geq$ mức đã ký.
test_executePartialChunk_revert_belowMinFill	Khớp dưới ngưỡng tối thiểu mỗi nhip bị từ chối.
test_executePartialChunk_revert_cancelled	Khớp một lệnh đã bị hủy bị từ chối.
test_reentrantOutputToken_reverts	Token đọc gọi ngược vào hợp đồng giữa lúc trả tiền, bị khóa reentrancy chặn.
Giá và sàn output	
test_tamperStartPrice_rejected	Filler sửa giá khởi điểm, giao dịch bị từ chối vì chữ ký không khớp.
test_fillBelowMinOutput_reverts	Nhip khớp trả ra dưới sàn theo tỉ lệ bị chặn.
test_feeOnTransferOutput_belowFloor_reverts	Token output thu phí khiến người dùng nhận dưới sàn bị chặn (đo số thực nhận).
test_feeOnTransferOutput_aboveFloor_creditsActualReceived	Người dùng được ghi đúng số thực nhận, không phải số danh nghĩa.
FillAuction — cọc và phạt	
test_register_success / _refundsExcess	Đăng ký đặt cọc thành công; phần gửi thừa được hoàn lại.
test_register_revert_*	Các điều kiện đăng ký: quá hạn, trùng, thiếu cọc, duy nhất Reactor được gọi...
test_slash_success / _revert_tooEarly	Phạt filler bỏ dở sau thời hạn; chặn phạt quá sớm.
test_onFillSuccess_fullCommitment_refundsFullStake	Giao đúng phần đã cam kết được hoàn toàn bộ cọc.

Tên hàm test	Kịch bản
test_H1_loserReclaimsStake_andCannotBeSlashed	Filler thua cuộc đua lấy lại đủ cọc và không thể bị phạt.
test_H2_cancelledOrder_notSlashable_reclaimable	Lệnh bị người dùng hủy: filler không bị phạt, lấy lại cọc.
test_33_invalidatedNonce_notSlashable_butReleasable	Nonce bị vô hiệu: filler không bị phạt, lấy lại cọc.
test_frontRunRace_loserReclaimsFullStake	Bị filler khác giành phần, vẫn hoàn tất phần dư và hoàn cọc đúng tỉ lệ.
test_snipeSmallChunk_fullyRefunded	Khớp phần nhỏ nhưng đúng cam kết vẫn được hoàn đủ cọc.
test_minFillBps_blocksDustFill	Khớp quá vụn (dưới ngưỡng) bị chặn.
RegistrationForgery	Filler không thể khai gian cỡ lệnh để mua cọc rẻ và bảo đảm hoàn 100%.
invariant_solveny	Bất biến: số dư hợp đồng luôn bằng tổng cọc đang giữ cộng tổng chờ rút.
test_population_honestAndMev_invariantsHold	Kịch bản nhiều lệnh trộn filler trung thực và filler xấu, kiểm các bất biến toàn cục.
Chống trích xuất giá trị bằng MEV	
test_lateFill_skimsDecaySpread_butStaysAboveFloor	Bot MEV khớp muộn thu được phần chênh do giá giảm, nhưng vẫn trên sàn.
test_lateFill_belowFloor_reverts	Bot MEV chờ tới khi giá dưới sàn, giao dịch bị từ chối.
test_registerThenAbandon_isSlashed_noProfit	Đăng ký rồi bỏ dở: bị phạt, không thu được lợi.
FallbackExecutor — cơ chế dự phòng	
test_fallback_swapsSuccessfully_viaRouter	Dự phòng swap nốt phần dư qua một sàn trong danh sách trắng.
test_fillAfterFallback_reverts	Sau khi dự phòng chạy, không thể khớp thường nữa (chống tiêu hai lần).
test_fallback_revert_belowSignedFloor	Kết quả dự phòng dưới mức đã ký bị từ chối.
test_fallback_revert_tooEarly/_expired	Chạy dự phòng sai thời điểm bị chặn.
test_fallback_revert_routerNotAllowed	Gọi tới sàn ngoài danh sách trắng bị từ chối.

Tên hàm test	Kịch bản
test_fallback_refundsUnc onsumedInput	Phần tài sản vào không dùng hết được hoàn lại cho người dùng.
Giao dịch xuyên chuỗi	
test_fillSlot_then_withd raw_paysFiller	Luồng chính: nhận slot rồi rút, filler nhận đúng tài sản.
test_fillSlot_secondTime _revertsSlotAlreadyFille d	Một slot không thể bị khớp hai lần.
test_fillSlot_invalidPro of_reverts	Bằng chứng Merkle sai bị từ chối trước khi đùng tài sản.
test_fillSlot_invalidSwa pperSig_reverts	Chữ ký người dùng sai bị từ chối.
test_fillSlot_zeroSafety Deposit_reverts	Đặt cọc an toàn bằng 0 bị chặn (chống grief- ing).
test_cancel_afterExpiry_ refundsSwapper_andPaysSw apperDeposit	Hết hạn: hoàn tài sản cho người dùng kèm tịch thu cọc an toàn của filler.
test_reopenSlot_afterCan cel	Slot bị kẹt sau khi hủy có thể được mở lại cho lượt mới.
test_T2geqT1_swapperTake sBothLegs	Bảo đảm điều kiện thời gian T2 sớm hơn T1; vi phạm khiến filler mất cả hai đầu.
Các thư viện và hàm cấu hình	
DecayCursorLib, DynamicSta keLib, RemainingLib, Scaled OutputLib	Kiểm các thư viện tính giá theo block, tính cọc, theo dõi phần còn lại.
test_set*_revert_notOwne r/_badBucket	Kiểm quyền và giới hạn của các hàm cấu hình (duy nhất chủ hợp đồng, đúng phạm vi).

D. KIẾN THỨC NỀN TẢNG

Phụ lục này trình bày các khái niệm nền tảng được nhắc tới trong phần thân. Đây là kiến thức chung, không phải đóng góp của đề án, nên được đưa xuống đây; phần thân tóm tắt ngắn và trở về các mục dưới.

D.1 Blockchain và hợp đồng thông minh

Blockchain là một sổ cái phân tán, ghi lại các giao dịch theo từng khối (block) nối tiếp nhau và được nhiều máy cùng giữ bản sao, khiến dữ liệu đã ghi thực tế không thể thay đổi. Ethereum là một blockchain [6] cho phép chạy *hợp đồng thông minh* (smart contract) - những đoạn mã được triển khai lên chuỗi và tự động thực thi theo đúng logic đã viết, không ai can thiệp được. Có hai loại tài khoản: tài khoản do người dùng kiểm soát bằng khóa riêng, và tài khoản hợp đồng. Mỗi giao dịch làm thay đổi trạng thái chuỗi đều tốn một khoản phí gọi là phí gas. Token ERC-20 là một chuẩn chung để tạo các đồng tiền số trên Ethereum, quy định các hàm cơ bản như chuyển tiền và cấp quyền chi tiêu.

D.2 Nonce

Nonce là một con số dùng một lần, gắn với mỗi lệnh hoặc mỗi giao dịch, nhằm chống việc phát lại (replay). Nếu không có nonce, một lệnh đã ký có thể bị gửi đi nhiều lần để thực hiện lặp lại ngoài ý muốn của người ký. Trong đề án, mỗi lệnh mang một nonce riêng; người dùng cũng có thể chủ động vô hiệu một nonce để hủy hiệu lực của lệnh mang nonce đó.

D.3 Cây Merkle

Cây Merkle [7] là một cấu trúc cho phép chứng minh “một phần tử thuộc một tập hợp” mà không cần đưa ra cả tập hợp. Từ danh sách các phần tử, ta băm từng phần tử thành các “lá”, rồi liên tục băm từng cặp lại với nhau cho tới khi còn một giá trị duy nhất gọi là gốc (root). Để chứng minh một phần tử nằm trong cây, cần đưa ra phần tử đó cùng một số ít giá trị băm trên đường từ lá tới gốc (gọi là bằng chứng Merkle); người kiểm tra băm dần lên và so với gốc. Nhờ đó, việc lưu và ký một giá trị gốc là đủ để xác thực nhiều phần tử - đây là cách đề án cấp quyền khớp cho từng slot xuyên chuỗi mà mỗi lệnh được ký một lần duy nhất. Một lưu ý an toàn: để chống tấn công tiền ảnh thứ hai (second-preimage), trong đó một nút trong của cây bị trình ngược lại như thể là một lá, đề án băm mỗi lá *hai lần* theo chuẩn của thư viện OpenZeppelin [4]; nhờ đó tiền ảnh của một lá luôn dài 32 byte, khác với 64 byte của một nút trong, nên hai loại không thể bị nhầm lẫn.

D.4 Hợp đồng khóa thời gian theo mã băm (HTLC)

Hợp đồng khóa thời gian theo mã băm (Hashed Timelock Contract) [3] là một cơ chế khóa tài sản dựa trên hai điều kiện: một khóa băm và một mốc thời gian. Người ta chọn một bí mật S , tính khóa băm $H = \text{băm}(S)$, rồi khóa tài sản với hai điều kiện: ai đưa ra đúng S được rút tài sản; nếu quá mốc thời gian mà chưa có bên nào rút, tài sản được hoàn về người gửi. Cơ chế này cho phép hai bên ở hai chuỗi khác nhau trao đổi tài sản một cách an toàn: khi một bên tiết lộ S để nhận tài sản ở chuỗi này, bên kia đọc được S và dùng nó để nhận tài sản ở chuỗi kia. Nhờ vậy, hai bên hoặc cùng nhận được tài sản, hoặc cùng được hoàn lại, mà không cần một bên trung gian giữ tiền.

D.5 Chuẩn ký EIP-712 và Permit2

EIP-712 là chuẩn ký dữ liệu có cấu trúc trên Ethereum. Thay vì ký một chuỗi byte khó đọc, người dùng ký một cấu trúc dữ liệu có tên trường rõ ràng, nên ví có thể hiển thị đúng nội dung sắp ký, giảm nguy cơ ký nhầm nội dung độc hại. Bản tin ký còn gắn với một “miền” (domain) gồm tên ứng dụng, mã chuỗi và địa chỉ hợp đồng, để một chữ ký dùng cho ứng dụng này không dùng lại được cho ứng dụng khác.

Permit2 là một hợp đồng cho phép người dùng cấp quyền chi tiêu token một lần, sau đó các hợp đồng được ủy quyền có thể kéo token tới đúng địa chỉ đích mà không cần người dùng ký lại từng lần. Đồ án dùng Permit2 để kéo tài sản của người dùng vào đúng lúc một phần lệnh được khớp, nhờ đó người dùng không phải khóa tài sản trước và không tốn phí gas khi tạo lệnh.

D.6 Đấu giá kiểu Hà Lan

Đấu giá kiểu Hà Lan (Dutch auction) là hình thức đấu giá ngược: giá bắt đầu ở mức cao rồi giảm dần theo thời gian, và người mua đầu tiên thấy mức giá đủ tốt sẽ chốt giao dịch. Khác với đấu giá tăng giá thông thường, ở đây không có việc các bên trả giá lên nhau; thị trường tự tìm ra mức giá mà tại đó có người sẵn sàng mua. Trong đồ án, giá của lệnh giảm dần theo từng block, và filler nào thấy mức giá hiện tại đủ để có lãi sẽ tham gia khớp.

D.7 MEV và tấn công chèn lệnh

MEV (Maximal Extractable Value) [2], [8] là phần giá trị mà những bên có quyền sắp xếp thứ tự giao dịch trong một block (như thợ đào hoặc người xây block) có thể trích xuất bằng cách thêm, bỏ hoặc đổi thứ tự giao dịch. Một dạng phổ biến là tấn công “kẹp” (sandwich): khi thấy một giao dịch mua lớn sắp được thực hiện trên một hồ thanh khoản, kẻ tấn công chèn một lệnh mua ngay trước để đẩy giá lên,

để nạn nhân mua ở giá cao, rồi bán ra ngay sau đó để hưởng chênh lệch. Việc định giá trước theo thời gian như trong đấu giá kiểu Hà Lan giúp giảm bớt loại tấn công này, vì mức giá không còn được quyết định tức thời tại thời điểm giao dịch.

D.8 CREATE2

CREATE2 là một opcode của EVM (được giới thiệu trong EIP-1014) cho phép triển khai hợp đồng tới một địa chỉ xác định trước mà không cần thực sự triển khai. Địa chỉ được tính từ bốn thành phần: tiền tố cố định `0xff`, địa chỉ hợp đồng triển khai, một giá trị `salt` tùy chọn, và hàm băm của bytecode hợp đồng:

$$\text{addr} = \text{keccak256}(0\text{xff} \parallel \text{deployer} \parallel \text{salt} \parallel \text{keccak256}(\text{bytecode}))[12:] \quad (\text{D.1})$$

Tính chất này cho phép hai bên tính toán và đồng thuận về địa chỉ hợp đồng ký quỹ *trước* khi nó được triển khai. Trong đề án, `EscrowSrcFactory` và `EscrowDstFactory` dùng CREATE2 để tạo các hợp đồng ký quỹ theo slot: `salt` được tính từ thông tin lệnh và chỉ số slot, đảm bảo địa chỉ hợp đồng là duy nhất và có thể xác minh độc lập bởi cả hai bên mà không cần giao tiếp trước.

D.9 Các thư viện OpenZeppelin sử dụng

Đề án sử dụng thư viện OpenZeppelin Contracts [4] — tập hợp các thành phần hợp đồng thông minh đã được kiểm toán và được dùng rộng rãi trong thực tế. Bảng D.1 liệt kê các thư viện được dùng trực tiếp.

Bảng D.1: Các thư viện OpenZeppelin được sử dụng trong đề án.

Thư viện	Vai trò trong đề án
MerkleProof	Xác minh bằng chứng Merkle cho từng slot xuyên chuỗi; băm lá hai lần để chống tấn công tiền ảnh thứ hai.
ECDSA	Phục hồi địa chỉ người ký từ chữ ký EIP-712 để xác thực chữ ký của người dùng và cosigner.
SafeERC20	Bọc các lần gọi <code>transfer/transferFrom</code> để phát hiện token trả về <code>false</code> thay vì <code>revert</code> , tránh mất tài sản thầm lặng.
ReentrancyGuard	Cung cấp modifier <code>nonReentrant</code> bảo vệ các hàm khớp lệnh và rút tài sản khỏi tấn công gọi lại.

E. BÁO CÁO RÀ SOÁT BẢO MẬT (TRUFY)

Dưới đây là toàn văn báo cáo rà soát bảo mật do Trufy thực hiện cho dự án NeutronX, tháng 6 năm 2026 [5]. Báo cáo ghi nhận 8 phát hiện (3 mức cao, 4 mức trung bình, 1 mức thấp); phân tích và phản hồi từng phát hiện được trình bày tại mục 4.4 của luận văn.



Smart Contract Audit Report

for

NeutronX

06-2026

Contents

1 Introduction	3
1.1 Project Summary	3
1.2 Vulnerability Summary	3
2 Findings	4
3 Detailed Results	5
3.1 cross-chain-slots-can-be-grief-locked-cheaply-because-fill-credentials-are-public-while-the-safety-bond-is-flat	5
3.2 non-standard-output-tokens-can-satisfy-nominal-accounting-while-underpaying-the-swapper	5
3.3 settlement-critical-watchers-can-silently-skip-historical-cross-chain-events-after-restarts-or-long-outages	6
3.4 cross-chain-nonce-is-documented-as-anti-replay-but-is-never-enforced-so-multiple-same-nonce-orders-can-remain-live-simultaneously	6
3.5 same-chain-collateral-can-still-be-manipulated-materially-because-it-relies-on-a-short-single-pool-twap-with-no-liquidity-sanity-checks	7
3.6 reopened-cross-chain-slots-cannot-be-retried-by-the-same-filler-address-on-the-destination-chain	7
3.7 fillauction-can-trap-eth-refunds-and-treasury-proceeds-for-non-payable-recipients	8
3.8 same-chain-fallback-support-is-narrower-than-order-creation-and-reactor-settlement-so-unsupported-tokens-silently-lose-the-fallback-safety-net	8
4 Appendix	9
4.1 Severity Definitions	9

1 Introduction

Trufy has been engaged by what to perform a security audit of the NeutronX smart contracts. The purpose of this audit is to achieve the followings:

- Ensure that smart contract functions work as intended.
- Identify possible vulnerabilities, which could be exploited by an attacker.
- Identify smart contract bugs, which might lead to unexpected behavior.
- Make recommendations to improve code safety and readability. As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage.

1.1 Project Summary

- Project Name: NeutronX
- Audit method: codex

1.2 Vulnerability Summary

Severity	# of Findings
High	3
Medium	4
Low	1
Informational	0
Optimization	0

2 Findings

The security audit identified 8 findings across different severity levels. Below is a summary of all identified issues:

1. cross-chain-slots-can-be-grief-locked-cheaply-because-fill-credentials-are-public-while-the-safety-bond-is-flat

Severity: high

Affected Elements: 3

2. non-standard-output-tokens-can-satisfy-nominal-accounting-while-underpaying-the-swapper

Severity: high

Affected Elements: 2

3. settlement-critical-watchers-can-silently-skip-historical-cross-chain-events-after-restarts-or-long-outages

Severity: high

Affected Elements: 2

4. cross-chain-nonce-is-documented-as-anti-replay-but-is-never-enforced-so-multiple-same-nonce-orders-can-remain-live-simultaneously

Severity: medium

Affected Elements: 3

5. same-chain-collateral-can-still-be-manipulated-materially-because-it-relies-on-a-short-single-pool-twap-with-no-liquidity-sanity-checks

Severity: medium

Affected Elements: 2

6. reopened-cross-chain-slots-cannot-be-retried-by-the-same-filler-address-on-the-destination-chain

Severity: medium

Affected Elements: 2

7. fillauction-can-trap-eth-refunds-and-treasury-proceeds-for-non-payable-recipients

Severity: medium

Affected Elements: 1

8. same-chain-fallback-support-is-narrower-than-order-creation-and-reactor-settlement-so-unsupported-tokens-silently-lose-the-fallback-safety-net

Severity: low

Affected Elements: 2

3 Detailed Results

3.1 cross-chain-slots-can-be-grief-locked-cheaply-because-fill-credentials-are-public-while-the-safety-bond-is-flat

Severity: high

Affected Code Elements (3)

node uint256 public constant MIN_SAFETY_DEPOSIT = 0.001 ether; contract/src/crosschain/EscrowSrcFactory.sol

Source Mapping

File: contract/src/crosschain/EscrowSrcFactory.sol

Lines: 148-148

Columns: -

Start Byte:

Length: bytes

Code

contract/src/crosschain/EscrowSrcFactory.sol

148 uint256 public constant MIN_SAFETY_DEPOSIT = 0.001 ether;

node EscrowSrc(escrow).initialize(value: msg.value)(hashlock, msg.sender, order.swapper, order.inputToken, amount, order.deadline); contract/src/crosschain/EscrowSrcFactory.sol

Source Mapping

File: contract/src/crosschain/EscrowSrcFactory.sol

Lines: 302-304

Columns: -

Start Byte:

Length: bytes

Code

contract/src/crosschain/EscrowSrcFactory.sol

```
302 EscrowSrc(escrow).initialize(value: msg.value){  
    hashlock, msg.sender, order.swapper, order.inputToken, amount,  
304 order.deadline  
304     );
```

node slots: slots.rows.map(s => ({ index: s.slot_index, hashlock: s.hashlock, proof: JSON.parse(s.proof) as string[], status: s.status, assignedFiller: s.assigned_filler, escrowAddr: s.escrow_addr,))) backend/src/services/crosschainService.ts

Source Mapping

File: backend/src/services/crosschainService.ts

Lines: 462-469

Columns: -

Start Byte:

Length: bytes

Code

backend/src/services/crosschainService.ts

```
462 slots: slots.rows.map(s => ({  
469     index:      s.slot_index,  
464     hashlock:   s.hashlock,  
465     proof:      JSON.parse(s.proof) as string[],  
466     status:     s.status,  
467     assignedFiller: s.assigned_filler,  
468     escrowAddr:  s.escrow_addr,
```

Description

`fillSlot()` is permissionless, binds the caller as the filler, and only requires a flat `0.001 ether` bond. The backend also returns `cosignerSig`, `swapperSig`, `hashlock`, and `proof` in the public order payload, so any observer can front-run a slot fill, lock the swapper's source assets into a fresh escrow, and abandon the destination leg until expiry.

Recommendation

Gate `fillSlot()` with an on-chain filler authorization tied to the slot or signed assignment, and scale the safety deposit with slot value or minimum output so griefing cannot be done for a near-constant cost. Avoid exposing claim credentials publicly before the intended filler is committed.

3.2 non-standard-output-tokens-can-satisfy-nominal-accounting-while-underpaying-the-swapper

Severity: high

Affected Code Elements (2)

node uint256 paid = _paidOutput[orderHash] + outputAmount;
_paidOutput[orderHash] = paid; contract/src/PartialFillReactor.sol

Source Mapping

File: contract/src/PartialFillReactor.sol

Lines: 173-174

Columns: -

Start Byte:

Length: bytes

Code

contract/src/PartialFillReactor.sol

```
173 uint256 paid = _paidOutput[orderHash] + outputAmount;  
174     _paidOutput[orderHash] = paid;
```

node IERC20(order.info.outputToken).safeTransferFrom(msg.sender,
order.info.swapper, outputAmount); contract/src/PartialFillReactor.sol

Source Mapping

File: contract/src/PartialFillReactor.sol

Lines: 180-180

Columns: -

Start Byte:

Length: bytes

Code

contract/src/PartialFillReactor.sol

```
180 IERC20(order.info.outputToken).safeTransferFrom(msg.sender,  
order.info.swapper, outputAmount);
```

Description

'executePartialChunk()' increments '_paidOutput' using the nominal 'outputAmount' before transferring the output token and never measures how many tokens the swapper actually receives. Fee-on-transfer or balance-distorting tokens can therefore satisfy reactor accounting while crediting the recipient less than the nominal amount.

Recommendation

Measure the swapper's output-token balance delta around the transfer and account for the actual received amount, or reject non-standard output tokens explicitly. Apply the same received-amount invariant to the cross-chain payout path.

3.3 settlement-critical-watchers-can-silently-skip-historical-cross-chain-events-after-restarts-or-long-outages

Severity: high

Affected Code Elements (2)

function **getCheckpoint**

backend/src/db/checkpoint.ts

Source Mapping

File: backend/src/db/checkpoint.ts

Lines: 15-20

Columns: -

Start Byte:

Length: bytes

Code

backend/src/db/checkpoint.ts

```
15 export async function getCheckpoint(name: string, currentBlock: number):  
    Promise<number> {  
20     const { rows } = await db.query('SELECT last_block FROM indexer_state WHERE  
        name = $1', [name])  
17     if (rows.length > 0) return Number(rows[0].last_block)  
        await db.query('INSERT INTO indexer_state (name, last_block) VALUES ($1,  
18         $2)', [name, currentBlock])  
19     return currentBlock  
20 }
```

function **resolveCheckpoint**

backend/src/db/checkpoint.ts

Source Mapping

File: backend/src/db/checkpoint.ts

Lines: 39-47

Columns: -

Start Byte:

Length: bytes

Code

backend/src/db/checkpoint.ts

```
39 export async function resolveCheckpoint(name: string, currentBlock: number,  
    label: string): Promise<number> {  
47     let last = await getCheckpoint(name, currentBlock)  
41     if (Math.abs(currentBlock - last) > MAX_CHECKPOINT_LAG) {  
42         console.warn(`${label}: checkpoint ${last} too far from tip  
        ${currentBlock} (chain reset?) - rewinding`)  
43         last = currentBlock  
44         await setCheckpoint(name, last)  
45     }  
46     return last  
47 }
```

Description

The checkpoint layer initializes unseen watchers at the current tip and also rewinds any lag above `1000` blocks directly to the current block. If the backend restarts after downtime or loses DB state, older `SlotFilled` or destination events are never replayed, which can leave `assigned\_filler` unset and cause secret reveal logic to refuse otherwise valid settlement.

Recommendation

Persist checkpoints conservatively and always replay missed ranges instead of seeding or rewinding to tip. If a safety cap is needed, page through historical logs in bounded chunks rather than discarding the backlog.

3.4 cross-chain-nonce-is-documented-as-anti-replay-but-is-never-enforced-so-multiple-same-nonce-orders-can-remain-live-simultaneously

Severity: medium

Affected Code Elements (3)

variable `_orders`

contract/src/crosschain/EscrowSrcFactory.sol

Source Mapping

File: contract/src/crosschain/EscrowSrcFactory.sol

Lines: 156-156

Columns: -

Start Byte:

Length: bytes

Code

contract/src/crosschain/EscrowSrcFactory.sol

```
156 mapping(bytes32 => OrderState) private _orders;
```

node

```
await db.query(` CREATE TABLE IF NOT EXISTS cc_orders (
  order_hash VARCHAR(66) PRIMARY KEY, swapper
  VARCHAR(42) NOT NULL, input_token VARCHAR(42) NOT
  NULL, input_amount VARCHAR(78) NOT NULL,
  output_token VARCHAR(42) NOT NULL, min_output
  VARCHAR(78) NOT NULL, deadline INTEGER NOT NULL,
  nonce VARCHAR(78) NOT NULL, num_slots SMALLINT NOT
  NULL, merkle_root VARCHAR(66) NOT NULL, cosigner_sig
  TEXT NOT NULL, swapper_sig TEXT, t2_expiry INTEGER
  NOT NULL, reactor_addr VARCHAR(42) NOT NULL,
  chain_a_id INTEGER NOT NULL, created_at TIMESTAMP
  DEFAULT NOW() ) `)
```

backend/src/services/crosschainService.ts

Source Mapping

File: backend/src/services/crosschainService.ts

Lines: 16-35

Columns: -

Start Byte:

Length: bytes

Code

backend/src/services/crosschainService.ts

```
16 await db.query(`
35   CREATE TABLE IF NOT EXISTS cc_orders (
18     order_hash    VARCHAR(66) PRIMARY KEY,
19     swapper       VARCHAR(42) NOT NULL,
20     input_token   VARCHAR(42) NOT NULL,
21     input_amount  VARCHAR(78) NOT NULL,
22     output_token  VARCHAR(42) NOT NULL,
23     min_output    VARCHAR(78) NOT NULL,
24     deadline      INTEGER NOT NULL,
25     nonce         VARCHAR(78) NOT NULL,
26     num_slots     SMALLINT NOT NULL,
27     merkle_root   VARCHAR(66) NOT NULL,
28     cosigner_sig  TEXT NOT NULL,
29     swapper_sig   TEXT,
30     t2_expiry     INTEGER NOT NULL,
31     reactor_addr  VARCHAR(42) NOT NULL,
32     chain_a_id    INTEGER NOT NULL,
33     created_at    TIMESTAMP DEFAULT NOW()
```

```
34    )
35    `)
```

```
await db.query(` INSERT INTO cc_orders (order_hash,
swapper, input_token, input_amount, output_token,
min_output, deadline, nonce, num_slots, merkle_root,
cosigner_sig, t2_expiry, reactor_addr, chain_a_id,
dst_chain_id) VALUES
node ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15) ON
CONFLICT (order_hash) DO NOTHING `, [structHash,
p.swapper, p.inputToken, p.inputAmount, p.outputToken,
p.minOutput, p.deadline, p.nonce, numSlots, merkleRoot,
cosignerSig, t2Expiry, reactorAddr, p.chainAId,
p.dstChainId])
```

backend/src/services/crosschainService.ts

Source Mapping

File: backend/src/services/crosschainService.ts

Lines: 351-358

Columns: -

Start Byte:

Length: bytes

Code

backend/src/services/crosschainService.ts

```
351 await db.query(`
352     INSERT INTO cc_orders
353     (order_hash, swapper, input_token, input_amount, output_token,
354     min_output,
355     deadline, nonce, num_slots, merkle_root, cosigner_sig, t2_expiry,
356     reactor_addr, chain_a_id, dst_chain_id)
357     VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15)
358     ON CONFLICT (order_hash) DO NOTHING
359     `, [structHash, p.swapper, p.inputToken, p.inputAmount, p.outputToken,
360     p.minOutput,
361     p.deadline, p.nonce, numSlots, merkleRoot, cosignerSig, t2Expiry,
362     reactorAddr, p.chainAId, p.dstChainId])
```

Description

The cross-chain order struct documents `nonce` as anti-replay, but the factory stores state only by `orderHash`, not by `(swapper, nonce)`. The backend schema likewise keys persisted orders by `order\_hash` only and inserts duplicates with the same nonce as long as the signed terms differ enough to change the hash. As a result, multiple same-nonce orders can coexist and be filled independently.

Recommendation

Track and consume `(swapper, nonce)` on-chain before accepting a first fill, and enforce a uniqueness constraint on `(swapper, nonce)` in the backend so a nonce cannot remain live across multiple orders.

3.5 same-chain-collateral-can-still-be-manipulated-materially-because-it-relies-on-a-short-single-pool-twap-with-no-liquidity-sanity-checks

Severity: medium

Affected Code Elements (2)

function toEthNotional

contract/src/libs/DynamicStakeLib.sol

Source Mapping

File: contract/src/libs/DynamicStakeLib.sol

Lines: N/A

Columns: -

Start Byte:

Length: bytes

Code

contract/src/libs/DynamicStakeLib.sol

```
1 function toEthNotional(
2     uint256 fillAmount,
3     address inputToken,
4     uint24 feeTier,
5     address weth,
6     address uniV3Factory,
7     uint32 twapWindow
8 ) internal view returns (uint256) {
9     if (uniV3Factory == address(0) || weth == address(0) || inputToken ==
weth) {
10         return fillAmount;
11     }
12     address pool = IUniswapV3Factory(uniV3Factory).getPool(inputToken,
weth, feeTier);
13     require(pool != address(0), "no twap pool");
14
15     uint32[] memory secondsAgos = new uint32[](2);
16     secondsAgos[0] = twapWindow;
17     secondsAgos[1] = 0;
18     (int56[] memory tickCumulatives, ) =
IUniswapV3PoolOracle(pool).observe(secondsAgos);
19
20     int56 delta = tickCumulatives[1] - tickCumulatives[0];
21     int24 meanTick = int24(delta / int56(uint56(twapWindow)));
22     if (delta < 0 && (delta % int56(uint56(twapWindow)) != 0)) meanTick-
-;
23
24     uint160 sqrtPriceX96 = TickMath.getSqrtPriceAtTick(meanTick);
25     return _quoteWeth(sqrtPriceX96, fillAmount, inputToken, weth);
26 }
```

node uint32 constant TWAP_WINDOW = 60;

contract/script/Deploy.s.sol

Source Mapping

File: contract/script/Deploy.s.sol

Lines: 25-25

Columns: -

Start Byte:

Length: bytes

Code

contract/script/Deploy.s.sol

```
25 uint32 constant TWAP_WINDOW = 60;
```

Description

Collateral sizing reads a single Uniswap V3 pool chosen by `(inputToken, WETH, feeTier)`, computes a mean tick over the configured lookback, and converts that directly into ETH notional with no liquidity floor, secondary oracle, or volatility sanity check. The deployment script configures only a 60-second window, so a manipulator who can distort that one pool long enough can materially underprice the required bond.

Recommendation

Use a longer observation window plus liquidity and deviation guards, or cross-check against an independent oracle before accepting the TWAP. Also set a non-zero `minCollateral` during deployment so manipulated quotes cannot drive stake close to zero.

3.6 reopened-cross-chain-slots-cannot-be-retried-by-the-same-filler-address-on-the-destination-chain

Severity: medium

Affected Code Elements (2)

node

bytes32 salt = _salt(orderHash, slotIndex, _attempt[orderHash][slotIndex]);

contract/src/crosschain/EscrowSrcFactory.sol

Source Mapping

File: contract/src/crosschain/EscrowSrcFactory.sol

Lines: 294-294

Columns: -

Start Byte:

Length: bytes

Code

contract/src/crosschain/EscrowSrcFactory.sol

294 bytes32 salt = _salt(orderHash, slotIndex, _attempt[orderHash][slotIndex]);

node

bytes32 salt = keccak256(abi.encodePacked(hashlock, msg.sender));

contract/src/crosschain/EscrowDstFactory.sol

Source Mapping

File: contract/src/crosschain/EscrowDstFactory.sol

Lines: 66-66

Columns: -

Start Byte:

Length: bytes

Code

contract/src/crosschain/EscrowDstFactory.sol

66 bytes32 salt = keccak256(abi.encodePacked(hashlock, msg.sender));

Description

Source-side retries increment an `attempt` counter and derive a fresh source escrow salt, but the destination factory derives its CREATE2 salt only from `hashlock` and the filler address. Because the hashlock is reused across attempts, the same filler always collides with the previously deployed destination escrow address and cannot retry with the same wallet.

Recommendation

Include a retry dimension such as the source attempt counter or a backend-signed retry nonce in the destination CREATE2 salt, and propagate that retry identifier through watcher validation so a reopened slot can be redeployed by the same filler address.

3.7 fillauction-can-trap-eth-refunds-and-treasury-proceeds-for-non-payable-recipients

Severity: medium

Affected Code Elements (1)

function withdraw

contract/src/FillAuction.sol

Source Mapping

File: contract/src/FillAuction.sol

Lines: 277-283

Columns: -

Start Byte:

Length: bytes

Code

contract/src/FillAuction.sol

```
277 function withdraw() external nonReentrant {
283     uint256 amount = pendingReturns[msg.sender];
279     require(amount > 0, "nothing to withdraw");
280     pendingReturns[msg.sender] = 0;
281     (bool ok,) = msg.sender.call{value: amount}("");
282     require(ok, "transfer failed");
283 }
```

Description

`FillAuction` accrues ETH liabilities in `pendingReturns`, but `withdraw()` always sends funds directly to `msg.sender` and offers no alternate recipient or fallback crediting path. If the owed address is a non-payable contract, every withdrawal attempt reverts and the ETH remains practically unrecoverable for that recipient.

Recommendation

Replace the direct push-only withdrawal with a pull-payment function that accepts a payable recipient parameter, or credit failed sends into a secondary claimable balance that can be withdrawn to another address.

3.8 same-chain-fallback-support-is-narrower-than-order-creation-and-reactor-settlement-so-unsupported-tokens-silently-lose-the-fallback-safety-net

Severity: low

Affected Code Elements (2)

variable **TOKEN_META**

backend/src/watcher/fallbackWatcher.ts

Source Mapping

File: backend/src/watcher/fallbackWatcher.ts

Lines: 45-52

Columns: -

Start Byte:

Length: bytes

Code

backend/src/watcher/fallbackWatcher.ts

```
45 const TOKEN_META: Record<string, { decimals: number; symbol: string }> = {
52   '0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2': { decimals: 18, symbol:
    'WETH' },
47   '0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48': { decimals: 6, symbol:
    'USDC' },
48   '0xdAC17F958D2ee523a2206206994597C13D831ec7': { decimals: 6, symbol:
    'USDT' },
49   '0x6B175474E89094C44Da98b954EedeAC495271d0F': { decimals: 18, symbol: 'DAI'
    },
50   '0x2260FAC5E5542a773Aa44fBcF7C193bc2C599': { decimals: 8, symbol:
    'WBTC' },
51   '0x514910771AF9Ca656af840dff83E8264EcF986CA': { decimals: 18, symbol:
    'LINK' },
52 }
```

node **if (!tokenIn || !tokenOut) { console.warn('Unknown token: \${order.input_token} / \${order.output_token}') continue }**

backend/src/watcher/fallbackWatcher.ts

Source Mapping

File: backend/src/watcher/fallbackWatcher.ts

Lines: 209-212

Columns: -

Start Byte:

Length: bytes

Code

backend/src/watcher/fallbackWatcher.ts

```
209 if (!tokenIn || !tokenOut) {
212     console.warn('Unknown token: ${order.input_token} /
    ${order.output_token}')
211     continue
212 }
```

Description

Same-chain orders and reactor settlement are generic over ERC-20s, but the fallback watcher hardcodes metadata for only six tokens and aborts when either side of a pair is absent. Orders for other tokens can still exist and be partially filled, but they lose the fallback completion path near expiry because the watcher cannot quote or execute them.

Recommendation

Source token metadata dynamically from the order/token registry instead of a fixed map, or reject unsupported tokens at order-creation time so the system does not advertise fallback support it cannot execute.

4 Appendix

4.1 Severity Definitions

High

This level vulnerabilities are significant security issues that could potentially be exploited to cause considerable damage. They require immediate attention and should be fixed as soon as possible.

Medium

This level vulnerabilities are hard to exploit but very important to fix, they carry an elevated risk of smart contract manipulation, which can lead to critical-risk severity.

Low

This level vulnerabilities should be fixed, as they carry an inherent risk of future exploits, and hacks which may or may not impact the smart contract execution.

Informational

This level vulnerabilities can be ignored. They are code style violations and informational statements in the code. They may not affect the smart contract execution.

Optimization

Gas Optimization findings refer to exhibits that do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction on the total gas cost of a transaction.